Technical Analysis of `memvar.h`: A History-Tracking Value Wrapper for C++20

Rel. 1.3.1

October 11, 2026

Abstract

This report analyses `memvar.h`, a single-header C++20 library that defines two variable wrappers: `mv::memvar<T>`, which behaves like an ordinary variable of type `T` but remembers a bounded number of its previous values, and `mv::memvarTimed<T, Time, Clock>`, which additionally stamps every value with the time at which it was written. It describes the *current* version of the header, in which both classes share one implementation, `memvar<T, Policy>`, and every history entry of a timed variable holds its value and its time tag together in a single container. The report is based exclusively on the code of the header and of the project's unit tests (comments were ignored); every code example and every measurement quoted here was compiled and executed against the header as delivered, with GCC 13.3 and libstdc++ on x86-64 Linux. The 47 project unit tests that do not require a one-billion-entry history pass in the two checked safety modes (46 in the default mode, where one SafeRef test does not exist), also under AddressSanitizer and UndefinedBehaviorSanitizer.

The report describes the motivation and the type-constraint system, the data layout and the write/read paths of both classes, the free operators and the stream-output helpers, the build-time switch `MEMVAR_SAFEREF_CHECK` that selects how dangling references into the history are handled, and gives complete, runnable usage examples. A dedicated section describes the unit-test suite, in particular the new tests that inject an allocation failure and prove that a failed write changes nothing; three of them fail against the former two-deque header (rel1.2.x or older), which is how the defect that motivated the redesign was demonstrated. The report then documents edge cases and closes with a catalogue of limitations, constraints and open issues: the failed-write defect of `memvarTimed` (issue I1) is resolved, while, for example, the default build still gives no protection against references that outlive their history slot. Prerequisites are working knowledge of C++20 templates, concepts and the standard containers.

Contents

1 Introduction and Motivation **4**
- 1.1 The problem . . . 4
- 1.2 Design goals . . . 4
- 1.3 Anatomy of the header . . . 4
- 1.4 Methodology and scope . . . 5

2 Type-Constraint System **6**
- 2.1 The building blocks . . . 6
- 2.2 Which types are accepted . . . 7
- 2.3 The auxiliary concepts used by the operators . . . 7

3 The `memvar<T, Policy>` Class **7**
- 3.1 Public interface at a glance . . . 7
- 3.2 Storage and data layout . . . 8
  - 3.2.1 What one history entry is . . . 8

3.2.2 Data members and the write path . . . 9
3.2.3 Index convention . . . 10
3.3 Construction and special members . . . 11
3.4 The write path . . . 11
3.4.1 Increment and decrement . . . 12
3.5 Read path . . . 12
3.6 Printing . . . 13
3.7 clearHistory . . . 13
3.8 Safe access to the current value: `cref()` and `SafeRef` . . . 13
3.8.1 The hazard . . . 13
3.8.2 The three modes . . . 14
3.8.3 How the check works . . . 15
3.8.4 What the handle does and does not protect . . . 15

4 The `memvarTimed<T, Time, Clock>` Class 16
4.1 Purpose and template parameters . . . 16
4.2 State and invariants . . . 17
4.3 The write path . . . 17
4.4 Reading time tags . . . 18
4.5 Overrides and name hiding . . . 18
4.6 What the single-deque design changed . . . 18
4.7 Choice of Time and Clock . . . 18

5 Free Functions and Namespace-Scope Operator Overloads 19
5.1 Why a friend struct and a concept . . . 19
5.2 The generated operators . . . 20
5.3 Stream output . . . 22
5.4 Unicode output helpers . . . 22
5.5 Other free functions . . . 23

6 Compilation and Compilation Flags 23
6.1 Language and library requirements . . . 23
6.2 Build configuration of the project . . . 23
6.3 Rules that follow from the code . . . 24
6.4 Run-time cost . . . 24

7 Complete Usage Examples 25
7.1 Basic use: history, indexing, printing, clearing . . . 25
7.2 Operators: arithmetic, comparison, compound assignment . . . 26
7.3 Time-tagged variables . . . 28
7.4 Big integers: a Fibonacci recurrence . . . 29
7.5 A realistic pattern: a sensor filter . . . 30
7.6 Dangling-reference protection in the three modes . . . 31
7.7 Strings and Unicode output . . . 32

8 Edge Cases 33
8.1 Capacity and index errors . . . 35
8.2 Assignment edge cases . . . 36
8.3 Numeric edge cases . . . 36
8.4 Reference-handle edge cases . . . 36
8.5 Failed writes: the strong guarantee . . . 37
8.6 Consequences of the single-deque design . . . 38

8.7 Stream operator lookup 40
8.8 Unusable but accepted types 41
8.9 Containers and storage of memvars 41

9 The Unit-Test Suite **41**
9.1 Structure and results 41
9.2 Injecting an allocation failure 42
9.3 The new tests 43
9.4 Gaps and fragilities of the suite 44

10 Limitations, Constraints, and Open Issues **45**
10.1 Constraints of the design 45
10.2 Summary of open issues 45
10.3 Details 46

11 Conclusion **50**

1 Introduction and Motivation

1.1 The problem

Many programs need, besides the current value of a variable, a short memory of what the variable held before: a control loop that needs the previous sample to compute a difference, a recurrence relation such as the Fibonacci numbers whose next term depends on the two previous ones, a monitoring routine that must detect that a reading has jumped, or a debugging session in which the last few assignments to a state variable are of interest. The usual solutions are a hand-written ring buffer next to the variable, or a pair of "current" and "previous" variables that are shifted by hand at every update. Both are error-prone because the bookkeeping is separate from the variable and must be repeated at every assignment site.

`memvar.h` moves that bookkeeping inside the variable. A `memvar<T>` is used almost like a `T`: it is assigned, compared, added, multiplied, incremented and streamed with the same syntax, but every assignment pushes the new value on a bounded history, and the older values stay available by age (`m(1)` is the value before the current one, `m(2)` the one before that, and so on).

1.2 Design goals

The following goals are implemented.

- **Value-like syntax.** Free operator templates, an implicit conversion `operator T()` and member assignment operators let a `memvar` appear in arithmetic and comparison expressions without explicit unwrapping.
- **Bounded memory.** The history length is fixed at construction (`historyCapacity_`) and the oldest value is discarded when the capacity is exceeded.
- **Restricted value types.** Only integral (except `bool`), floating-point, the six standard string types, and a designated big-integer type are accepted; the restriction is expressed as a C++20 concept (Section 2).
- **Optional time tagging.** `memvarTimed` derives from `memvar<T, Policy>`, a class template whose second parameter decides what one history entry is: the bare value, or the value together with a time tag. Both flavours share one write path.
- **Failed writes change nothing.** A write is a single insertion into a single container, so an exception (for instance `std::bad_alloc`) leaves the variable, its history and its time tags exactly as they were (Section 3).
- **Cheap, optional protection against dangling references.** The accessor `cref()` can return a checked handle (`SafeRef`) whose cost is one load and one comparison, or a plain `const T&` with no overhead at all; the choice is made at build time (Section 3.8).
- **Single header, no external dependency** apart from the standard library (and, for the big-integer case, a forward declaration of `bip::bigint`).
- **No thread safety.** Nothing in the header is synchronised.

1.3 Anatomy of the header

The header (558 lines) has the following parts, listed in the order in which they appear. Throughout the report `memvar<T>` is shorthand for `memvar<T, void>`, the plain flavour (`void` is the default value of the second template parameter).

The class relationship is simple:

Part	Lines	Content
Configuration macros	37–49	`MEMVAR_SAFEREF_ABORT/THROWS/NO_CHECK`, default selection, `#error` on an invalid value
`mv::detail` (text)	54–106	UTF-8 encoder, `toUtf8<CharT>`, `throwIndexOutOfRange`
Global stream operators	108–117	`operator«` for `wstring`, `u16string`, `u32string`, `u8string`
Traits and concepts	119–149	`is_bigint`, `IsAnyOf`, `AnyStandardString`, `MemVarType`
Error support	151–172	`expired_history_reference`, abort path (only if checking is enabled)
`mv::detail` (entries)	174–202	`TimeTagPolicy`, `Stamped`, `SlotOf`; forward declaration of `memvar<T, Policy>`
Operator plumbing	204–228	`memvarAccess`, `IsMemvar`, `valueOf`, `MemvarOperands`
`memvar<T, Policy>`	230–435	the history-keeping variable, `SafeRef`
`memvarTimed`	437–500	the time-tagged variable
Free functions	502–510	`getHistoryValue`, `getHistoryTimedValue`
Operator templates	512–546	macro-generated binary, comparison and compound-assignment operators
Global stream operators	550–558	`operator«` for `memvar` and `memvarTimed`

Table 1: Layout of `memvar.h`.

Listing 1: Class relationship (simplified).

```
mv::memvar<T, Policy = void>              <- concrete, polymorphic (virtual
    dtor, virtual clearHistory)
   |  history entry ("Slot") = T                          when Policy is void
   |                          = detail::Stamped<T, tag>   otherwise
   ^
   | public, final
mv::memvarTimed<T, Time, Clock>           <- derives from memvar<T, detail::
    TimeTagPolicy<Time, Clock>>;
                                             adds memvarEpoch_, getTimeTag,
                                                 timed printing

mv::memvarAccess                          <- friend of memvar<T, P>; lets free
     operators read/write protected state
mv::memvar<T, P>::SafeRef                 <- nested, exists only if
    MEMVAR_SAFEREF_CHECK != NO_CHECK
```

An important consequence, examined in Section 4, is that `memvarTimed<int>` *is not* a `memvar<int>`: both are instances of the same template but with different second arguments, so they are unrelated types. Generic code must take `memvar<T, P>` or use the free operators.

1.4 Methodology and scope

The analysis is static (reading the code) plus dynamic (compiling and running probes). All numbers come from one machine (a virtual machine, GCC 13.3, libstdc++, `-std=c++20 -O3 -march=native -DNDEBUG` unless stated); sizes and allocation counts depend on the standard library implementation and are labelled as such. Statements that were derived by inspection only and not reproduced are marked *(by inspection)*. The header and the unit-test file are the versions delivered

with this report. The project's own unit tests (48 tests; 47 in the checked modes and 46 in the default mode are run, because `memVarTest.test_6` allocates a history of one billion entries) were re-run in all three `MEMVAR_SAFEREF_CHECK` modes and pass; the same binaries built with `-fsanitize=address,undefined` also pass, and so do all example programs of this report.

This report supersedes earlier ones written for the previous version of the header (rel.1.2.x or older), whose `memvarTimed` kept the values and the time points in two parallel `std::deques`. The differences that matter are collected in Section 4.6.

2 Type-Constraint System

2.1 The building blocks

Listing 2: Traits and concepts of the constraint system (memvar.h).

```
122 template <typename T>
123 struct is_bigint : std::false_type {};
124
125 template <>
126 struct is_bigint<bip::bigint> : std::true_type {};
127
128 template <typename T>
129 inline constexpr bool is_bigint_v = is_bigint<T>::value;
130
131 template <typename T, typename... Types>
132 concept IsAnyOf = (std::same_as<T, Types> || ...);
133
134 template <typename T>
135 concept AnyStandardString = IsAnyOf<std::remove_cvref_t<T>,
136                                     std::string,
137                                     std::wstring,
138                                     std::u8string,
139                                     std::u16string,
140                                     std::u32string>;
141
142 // Types allowed as memvar<T>'s T: integral (not bool), floating point,
143 // standard string, or bigint.
144 template <typename T>
145 concept MemVarType = (std::is_integral_v<std::remove_cvref_t<T>> &&
146                       !std::same_as<std::remove_cvref_t<T>, bool>) ||
147                       std::is_floating_point_v<std::remove_cvref_t<T>> ||
148                       AnyStandardString<T> ||
149                       is_bigint_v<std::remove_cvref_t<T>>;
```

`is_bigint<T>` / `is_bigint_v<T>`
: A trait that is false for every type except the explicit specialisation for `bip::bigint`. The specialisation needs only the *incomplete* type, which is why the header contains the forward declaration `namespace bip { class bigint; }` and does not include `bigint.h`. Note that `bigint.h` defines its own, identical trait `bip::is_bigint`; the two are unrelated.

`IsAnyOf<T, Types...>`
: A fold expression over `std::same_as`: true if `T` is exactly one of the listed types.

`AnyStandardString<T>`
: True for `std::string`, `std::wstring`, `std::u8string`, `std::u16string` and `std::u32string` (cv-qualifiers and references are removed first). It is used twice: as part of `MemVarType`, and as the discriminator that disables the arithmetic that makes no sense on strings (`-`, `*`, `/`, `++`, `-`).

`MemVarType<T>`

The constraint on `memvar`'s and `memvarTimed`'s first template parameter: an integral type other than `bool`, *or* a floating-point type, *or* a standard string, *or* a big-integer type.

2.2 Which types are accepted

Type `T`	`MemVarType<T>`	**Observed result of** `memvar<T>`
`int`, `long`, `unsigned`, `size_t`, `int8_t`	satisfied	works
`char`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`	satisfied (they are integral)	`memvar<char>` compiles; values print as characters
`float`, `double`, `long double`	satisfied	works
`std::string`, `wstring`, `u8string`, `u16string`, `u32string`	satisfied	works; `-`, `*`, `/`, `++`, `--` unavailable
`bip::bigint`	satisfied via `is_bigint`	works if `bigint.h` is included first
`bool`	rejected	"template constraint failure"
`std::string_view`, `const char*`, enums, `std::complex`, `std::pmr::string`	rejected	"template constraint failure"
`const int`	**satisfied** (cv-qualifiers are stripped)	hard error inside `std::deque` ("must have a non-const, non-volatile value_type")
`int&`	**satisfied** (references are stripped)	hard errors in the class body and in the allocator

Table 2: Effect of the constraint on representative types (acceptance verified with `static_assert`; diagnostics from GCC 13.3).

The last two rows show a mismatch between the concept and the class: the concept applies `std::remove_cvref_t` to `T`, but the class stores `std::deque<T>` with the unmodified `T`. A constraint that rejected cv-qualified and reference types would have produced a clear diagnostic at the point of use instead of an error inside the standard library (see issue I8).

2.3 The auxiliary concepts used by the operators

Three more concepts support the free operators of Section 5 and are described there: `IsMemvar<X>`, `MemvarOperands<Op, L, R>` and the helper function `valueOf`.

3 The `memvar<T, Policy>` Class

3.1 Public interface at a glance

`memvar` is declared as `template <MemVarType T, typename Policy = void>`. With the default it is the plain variable `memvar<T>`; with the policy `detail::TimeTagPolicy<Time, Clock>` it is the base class of `memvarTimed` (Section 4). The members below belong to both.

Member	Meaning
`memvar()`	value-initialised `T{}`, default capacity 10
`explicit memvar(const T&, capacityType = 10)`	initial value and history capacity (≥ 2, else `std::invalid_argument`)
`operator=(const T&), operator=(T&&), operator=(const memvar&)`	record a new value (the last one copies only the *current value* of the right-hand side)
`operator T(), operator()()`	current value, *by copy*
`operator()(i), operator[](i), valueAt(i)`	the value written i writes ago (0 = newest), by copy; `std::out_of_range` if $i < 0$ or $i \geq$ size
`getHistoryValue(i)`	`std::tuple<T,bool>`: the value and an error flag (always `false`); for a timed variable the tag is *not* part of it
`getHistoryMinMax()`	`std::tuple<T,T>` of the smallest and largest stored *value* (compared through a projection, so the tag of a timed entry plays no part)
`getHistorySize(), getHistoryCapacity(), isHistoryFull()`	current number of stored values, capacity, size $\geq$ capacity
`getMemVarHistory()`	`const auto&`: a reference to the internal `std::deque<Slot>`, i.e. `deque<T>` for a plain variable and `deque<detail::Stamped<T, tag>>` for a timed one
`++`, `-` (prefix/postfix)	not for string types; prefix returns the *new* value, postfix the *old* one, both by value
`printHistoryData(os, sep), printReverseHistoryData(os, sep)`	newest-to-oldest / oldest-to-newest dump
`clearHistory()` (virtual)	drop the whole history and restart from `T{}`; a failed clear leaves the history untouched
`Slot` (type alias)	the history entry type, `T` or `detail::Stamped<T, tag>`
`cref()`	read access to the current value; return type depends on `MEMVAR_SAFEREF_CHECK`

Table 3: Public members of `memvar<T>` (arithmetic and comparison operators are free functions, Section 5).

3.2 Storage and data layout

3.2.1 What one history entry is

The type of an entry is chosen at compile time from the second template parameter (Listing 3).

Listing 3: Tag policy, stamped entry and entry-type selection (memvar.h).

```
174 namespace detail {
175 // Tag policy of memvarTimed: how a time tag is produced.
176 template <typename Time, typename Clock>
177 struct TimeTagPolicy {
178   using tag_type = std::chrono::time_point<Clock, Time>;
179   static tag_type now() { return std::chrono::time_point_cast<Time>(Clock::now
        ()); }
180 };
181
182 // A history entry that carries a tag next to its value (timed variables).
183 template <typename T, typename Tag>
184 struct Stamped {
185   template <typename U>
186   Stamped(U&& v, const Tag t) : value(std::forward<U>(v)), tag(t) {}
187   T value;
```

```
  Tag tag;
};

// Entry type stored in the history: T itself, or Stamped<T, tag> for a policy.
template <typename T, typename Policy>
struct SlotOf { using type = Stamped<T, typename Policy::tag_type>; };
template <typename T>
struct SlotOf<T, void> { using type = T; };
}  // namespace detail

// Policy == void: a plain variable. Otherwise the policy supplies the type of
// a tag (tag_type) and a function that makes one (now()), and every history
// entry holds its value and its tag together, so they are inserted and
// evicted as one unit.
template <MemVarType T, typename Policy = void> class memvar;
```

- `TimeTagPolicy<Time, Clock>` says what a tag is (`tag_type`, a `time_point<Clock, Time>`) and how to make one (`now()`, the clock reading truncated to `Time`). There is no concept constraining a policy: any type with a `tag_type` and a static `now()` is accepted by the first-template-argument rules and fails with a hard error otherwise (issue I17).
- `Stamped<T, Tag>` is a plain pair `{value, tag}` with a converting constructor `Stamped(U&&, Tag)`.
- `SlotOf` yields the entry type: `T` for a `void` policy, otherwise a `Stamped` whose tag type is the policy's `tag_type`. It is exposed as the member alias `memvar::Slot`.

3.2.2 Data members and the write path

Listing 4: Members, entry access and the single write path of memvar (memvar.h).

```
protected:
  static constexpr capacityType minimumHistoryCapacity_ {2};
  static constexpr capacityType historyCapacityDefault_ {10};

  // Runs ahead of memo_'s construction (historyCapacity_ is declared first).
  static capacityType validateHistoryCapacity(const capacityType
      historyCapacity) {
    if ( historyCapacity >= minimumHistoryCapacity_ ) return historyCapacity;
    throw std::invalid_argument("ERROR: The history capacity must be " +
                                std::to_string(minimumHistoryCapacity_) + " at
                                    least, or more");
  }

  static const T& slotValue(const Slot& s) noexcept {
    if constexpr ( std::is_void_v<Policy> ) return s;
    else return s.value;
  }

  const capacityType historyCapacity_ {historyCapacityDefault_};
  std::deque<Slot> memo_ {};
#if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
  // writes ever made (construction is #1); only SafeRef::check() reads it
  capacityType writeCount_ {1};
#endif

  // Same-expression access only: the reference dies with the next write.
  const T& getValue() const { return slotValue(memo_.front()); }

  // The entry written i writes ago, bounds-checked.
```

```
397   const Slot& slot(const capacityType index) const {
398     if ( index >= 0 && index < getHistorySize() ) {
399       return memo_[static_cast<std::size_t>(index)];
400     }
401     detail::throwIndexOutOfRange(index, memo_.size());
402   }
403
404   // Inserts a new newest entry (value plus, for a policy, its tag). This is
405   // the only operation of a write that can throw, and it is a single
406   // insertion into a single container, so a failure changes nothing.
407   template <typename U>
408   void pushFront(U&& value) {
409     if constexpr ( std::is_void_v<Policy> ) memo_.emplace_front(std::forward<U
          >(value));
410     else memo_.emplace_front(std::forward<U>(value), Policy::now());
411   }
412
413   // Single write path; U is const T& (copy) or T (move).
414   template <typename U>
415   void setValue(U&& value) {
416     pushFront(std::forward<U>(value));
417     // nothing below can throw
418 #if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
419     ++writeCount_;
420 #endif
421     if ( getHistorySize() > historyCapacity_ ) memo_.pop_back();
422   }
```

The state of a `memvar` consists of:

- `historyCapacity_` (a `const capacityType`) – the maximum number of stored values. `capacityType` is `int64_t`; the code relies on its being signed (the bounds checks test `index >= 0`). The member is `const`, so a `memvar` can never be resized, and it is declared *before* `memo_` so that its initialiser (`validateHistoryCapacity`) runs, and can throw, before the deque is constructed.
- `std::deque<Slot> memo_` – the history itself. The *newest* entry is at the front (index 0) and the oldest at the back. A write is `emplace_front`; when the size exceeds the capacity the back element is removed with `pop_back`. The history therefore always holds the last $\min(\text{writes}, \text{capacity})$ entries. For a timed variable the entry is the value *and* its time tag, so no second container exists.
- `capacityType writeCount_` – present *only* when `MEMVAR_SAFEREF_CHECK` is not `NO_CHECK`. It counts the values ever written (construction counts as write number 1) and exists solely for `SafeRef::check()` (Section 3.8).
- The object is polymorphic (`virtual ~memvar()` and `virtual clearHistory()`), so it carries a pointer to a virtual table. Nothing on the write path is virtual any more.

Measured footprint with libstdc++ on x86-64 is given in Table 9; the history is conceptually a ring of `capacity` slots, but the implementation uses a deque and therefore inherits its allocation behaviour (see issue I5).

3.2.3 Index convention

Index 0 is always the current value; the highest valid index is `getHistorySize() - 1`. The following picture shows a `memvar<int>` with capacity 4 after the writes `0, 10, 20, 30, 40, 50, 60`:

Listing 5: History of a capacity-4 variable after seven writes (0 is the initial value).

```
index        0       1       2       3
```

```
memo_      [ 60  |  50  |  40  |  30 ]      <- 10, 20 and the initial 0
    were evicted
           newest                oldest
front <-- emplace_front(new)               pop_back() --> back
```

3.3 Construction and special members

Listing 6: Constructors, destructor and deleted special members (memvar.h).

```
292  memvar() : memvar(T{}) {}
293
294  explicit memvar(const T& value, const capacityType historyCapacity =
         historyCapacityDefault_) :
295    historyCapacity_(validateHistoryCapacity(historyCapacity)) {
296    pushFront(value);
297  }
298
299  virtual ~memvar() = default;
300
301  memvar(const memvar&) = delete;
302  memvar(memvar&&) = delete;
303  memvar& operator=(memvar&&) = delete;
```

- The default constructor delegates to the value constructor with `T{}` and the default capacity, so there is a single place that validates the capacity and stores the first value.
- The value constructor is `explicit`: `memvar<int> m = 5;` does not compile, `memvar<int> m(5);` and `memvar<int> m{5};` do.
- The value constructor stores the first entry with `pushFront` (and not with `setValue`, because the write counter already starts at 1). For a timed variable `pushFront` makes the tag too, so the derived constructor has nothing to initialise but the epoch, which it reads from the entry that was just stored (Section 4). If `pushFront` throws, the constructor fails as a whole and no object exists.
- Copy construction, move construction and move assignment are deleted (confirmed with `static_asserts` in the edge-case program of Section 8). Copy *assignment* exists and copies only the current value of the right-hand side as a new write; history and capacity are never copied. A consequence is that `a = std::move(b)` between two `memvars` does not compile, because the deleted `operator=(memvar&&)` is the best match.

3.4 The write path

All writes funnel through the protected template `setValue(U&&)` (Listing 4):

1. `pushFront(value)` constructs the new entry in place at the front of the deque. A plain variable emplaces just the value; a timed one emplaces the value and a tag obtained from `Policy::now()` (`U` is `const T&` for a copy or `T` for a move, so an rvalue assigned to a `memvar<std::string>` is moved, not copied);
2. if checking is enabled, `writeCount_` is incremented;
3. if the size now exceeds the capacity the oldest entry is dropped with `pop_back`.

Writes are amortised $O(1)$. The decisive property is the order of the steps: the only operation that can throw (allocation of a deque node, or the copy constructor of `T`) is step 1, and it is a *single* insertion into a *single* container. Steps 2 and 3 (an integer increment, a size comparison and removal of an element,

whose destructor is assumed not to throw) cannot fail. Hence a write either happens completely or not at all: the *strong exception guarantee*. Three consequences follow, all covered by unit tests (Section 9):

- a failed write on a full history does *not* evict the oldest value;
- a failed write does not count in `writeCount_`, so it does not age any outstanding `SafeRef`;
- for a timed variable, a value and its tag can never get out of step, because they are one object (Section 4).

The three assignment operators of Table 3 simply forward to `setValue`. The compound assignments (`+=` etc.) are free functions that first compute the new value and only then call `setValue`; so if the arithmetic throws (for example a `bigint` division by zero) nothing is recorded (Section 7). Note that for a timed variable the clock is read *before* the value is constructed (it is an argument of `emplace_front`), so the tag marks the start of the write, not its commit; for a large string the difference is the copy time.

3.4.1 Increment and decrement

Listing 7: Increment and decrement operators (memvar.h).

```
330   // ++mv, --mv return the new value; mv++, mv-- the old one.
331   T operator++() requires (!AnyStandardString<T>) {
332     setValue(static_cast<T>(getValue() + 1));
333     return getValue();
334   }
335   T operator--() requires (!AnyStandardString<T>) {
336     setValue(static_cast<T>(getValue() - 1));
337     return getValue();
338   }
339   T operator++(int) requires (!AnyStandardString<T>) {
340     T old {getValue()};
341     setValue(static_cast<T>(old + 1));
342     return old;
343   }
344   T operator--(int) requires (!AnyStandardString<T>) {
345     T old {getValue()};
346     setValue(static_cast<T>(old - 1));
347     return old;
348   }
```

All four are constrained with `requires (!AnyStandardString<T>)` on a non-template member function, a C++20 feature that makes the member *unavailable* rather than ill-formed for string types. The result is computed as `static_cast<T>(value ± 1)`, which is what keeps narrow types such as `unsigned char` well-defined (they are promoted to `int` for the addition and truncated back). Each of the four functions returns `T` *by value*, not `memvar&`; hence, unlike the built-in operators, `++m = 3` is not valid. The postfix forms copy the old value before writing and return it. For a signed integral `T` that is not narrower than `int`, incrementing the maximum value is signed overflow, which is undefined behaviour *(by inspection)*.

3.5 Read path

- `getValue()` (protected) returns the value of the front entry by reference through `slotValue`, a static helper that returns the entry itself for a plain variable and its `.value` member for a stamped one; it is the primitive used inside the library.
- `operator T()`, `operator()()` return `T` by value.
- `operator()(i)`, `operator[](i)`, `valueAt(i)` and `getHistoryValue(i)` all go through

the protected `slot(i)`, which performs the bounds check (`i >= 0 && i < size`) and throws `std::out_of_range` through the `[[noreturn]]` helper `detail::throwIndexOutOfRange`. Message: `Index 5 is out of bounds (size is 2)`. The time-tag accessors of `memvarTimed` use the same `slot(i)` and therefore report the same message.

- `getHistoryMinMax()` uses `std::ranges::minmax_element` on the deque with a projection to the value part of each entry and returns copies of the two values; strings are compared lexicographically, floating-point values with <.
- `getMemVarHistory()` returns a const reference to the whole deque: this is the only way to iterate without copying, and the reference is invalidated for the removed element by the next write. Its element type is `Slot`, so for a timed variable the caller sees `detail::Stamped` objects.

The free function `mv::getHistoryValue(const memvar<T, P>&, index)` returns the *value only* (it is `mv(index)`), whereas the member `getHistoryValue(index)` returns a `tuple<T,bool>`; the same name therefore has two different return types depending on how it is called.

3.6 Printing

`printHistoryData` and `printReverseHistoryData` call the private `print(os, separator, reverse)`, which writes `"[ "`, then each value followed by the separator, then `"]\n"`. With the default separator the output of a three-element history is `[ 3 2 1 ]`; there is a trailing separator before the closing bracket. Values are streamed with `operator«` found by ordinary unqualified lookup, so wide strings use the UTF-8 helpers of Section 5.

3.7 clearHistory

Listing 8: clearHistory of the base class (memvar.h).

```
359   // Restarts from T{}. The new entry is inserted before the old ones are
360   // erased, so a failure leaves the history untouched.
361   virtual void clearHistory() {
362     pushFront(T{});
363     memo_.erase(std::next(memo_.begin()), memo_.end());
364 #if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
365     // every outstanding SafeRef is now dangling: force them all to expire
366     writeCount_ += historyCapacity_;
367 #endif
368   }
```

`clearHistory()` (virtual) first inserts a value-initialised `T{}` at the front and *then* erases everything behind it. The order matters: if the insertion throws, the old history is still intact; the erasure cannot throw. When checking is on, `historyCapacity_` is added to `writeCount_` afterwards. The current value is therefore *reset to zero/empty string*, not kept; see issue I10. For a timed variable the inserted entry is stamped like any other and `memvarTimed::clearHistory()` makes its tag the new epoch (Section 4).

3.8 Safe access to the current value: `cref()` and `SafeRef`

3.8.1 The hazard

A reference to an element of the history is invalidated when that element is evicted. A caller who keeps `const T& r = m.getValue()` across enough writes reads a destroyed object. `getValue()` is protected for that reason, and the public accessor is `cref()`.

3.8.2 The three modes

`MEMVAR_SAFEREF_CHECK` must be defined to one of three values before the header is included (or on the compiler command line); any other value stops compilation with `#error`. When it is undefined the header selects `MEMVAR_SAFEREF_NO_CHECK`.

Macro value	Value	`cref()` returns	On an expired handle
`MEMVAR_SAFEREF_ABORT`	0	`SafeRef`	message to `stderr`, `std::abort()` (noexcept, out of line)
`MEMVAR_SAFEREF_THROWS`	1	`SafeRef`	throws `mv::expired_history_reference` (derives from `std::logic_error`)
`MEMVAR_SAFEREF_NO_CHECK`	2 (default)	`const T&`	nothing: undefined behaviour if the slot was evicted

Table 4: The three safety modes.

Which mode to use? Apply the following rules of thumb:

- `MEMVAR_SAFEREF_NO_CHECK` (the default): You might have UB in your code, but only if you access references to evicted items; therefore, use this mode if you are sure of never accessing such kind of references in your code.
- `MEMVAR_SAFEREF_THROWS`: If you fear to reference evicted items in your code, you can always throw an exception; this is the fastest and more efficient way to tackle such an event in production code.
- `MEMVAR_SAFEREF_ABORT`: Unlike throwing, aborting can be considered only in development stage, not in production.

Listing 9: SafeRef and cref() (memvar.h).

```
246 #if MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_NO_CHECK
247   // Non-owning handle to the current value at the time cref() was called.
248   // It remembers the write count at which its slot will be evicted and checks
249   // that against the live write count on each dereference, so a stale handle
250   // fails instead of reading reused deque storage. No allocation, no
251   // refcounting; a passing check costs one load and one compare. It must not
252   // outlive the memvar itself.
253   class SafeRef {
254   public:
255     const T& operator*() const noexcept(MEMVAR_SAFEREF_CHECK !=
          MEMVAR_SAFEREF_THROWS) {
256       check();
257       return *ptr_;
258     }
259     const T* operator->() const noexcept(MEMVAR_SAFEREF_CHECK !=
          MEMVAR_SAFEREF_THROWS) {
260       check();
261       return ptr_;
262     }
263     operator const T&() const noexcept(MEMVAR_SAFEREF_CHECK !=
          MEMVAR_SAFEREF_THROWS) {
264       return operator*();
265     }
266
267   private:
268     friend class memvar;
269     SafeRef(const T* ptr, const capacityType writeSeq, const memvar* owner)
          noexcept :
270       ptr_(ptr), expiryWriteCount_(writeSeq + owner->historyCapacity_), owner_
            (owner) {}
271
```

```
272     void check() const noexcept(MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_THROWS)
            {
273       if ( owner_->writeCount_ >= expiryWriteCount_ ) [[unlikely]] {
274 #if MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_THROWS
275         throw expired_history_reference();
276 #else
277         reportExpiredHistoryReferenceAndAbort();
278 #endif
279       }
280     }
281
282     const T* ptr_;
283     capacityType expiryWriteCount_;
284     const memvar* owner_;
285   };
286
287   SafeRef cref() const { return SafeRef(&getValue(), writeCount_, this); }
288 #else
289   const T& cref() const { return getValue(); }
290 #endif
```

3.8.3 How the check works

A `SafeRef` stores three things: a pointer to the slot, the owner `memvar*`, and `expiryWriteCount_`, computed at issue time as `writeCount_ + historyCapacity_`. Because a slot written as write number w is evicted exactly when write number $w +$ capacity happens, the handle is valid as long as the live `writeCount_` is below that precomputed value. Table 5 shows a capacity-3 variable.

Event	`writeCount_`	**History (newest first)**	**Handle on value 1**
construct with 1; `h = m.cref()`	1	1	valid
`m = 2`	2	2 1	valid
`m = 3`	3	3 2 1	valid
`m = 4`	4	4 3 2	**expired** (value 1 evicted)

Table 5: Expiry of a `SafeRef` issued at write number 1 with capacity 3.

`clearHistory()` adds the capacity to `writeCount_`, which pushes the counter past the expiry of every handle that was issued earlier. A write that throws does not touch `writeCount_` (the increment follows the insertion), so it cannot expire a handle either; this holds in both checked modes and is tested.

Two design points explain the cost. First, `check()` is one load of the owner's counter and one comparison, with `[[unlikely]]` on the failure branch; no allocation, no reference count. Second, the `ABORT` variant reports through a `[[noreturn]] noexcept` non-template function and all `SafeRef` accessors are declared `noexcept(MEMVAR_SAFEREF_CHECK != MEMVAR_SAFEREF_THROWS)`: a call site that cannot throw does not need unwinding tables or register spills, which is what keeps the checked read as cheap as a plain reference (Table 8). The `THROWS` variant gives up that property.

3.8.4 What the handle does and does not protect

- It protects against *slot eviction* (too many writes, or `clearHistory()`), and a handle *pins the slot*, not the "current value": after a new write the handle still designates the value it was issued for.
- It does *not* protect against destruction of the `memvar` itself (it holds a raw owner pointer).
- The conversion `operator const T&()` lets a handle be used where a `const T&` is expected, but implicit conversions do not take part in template argument deduction or member-operator

lookup; explicit dereference (`*h`) is the reliable form in such contexts (for instance `*h + "x"` with `std::string`).

- In the default mode there is no handle: `cref()` is `const T&`, and `auto c = m.cref();` silently makes a *copy* of the value, whereas in the checked modes the same line stores a `SafeRef` that can expire (issue I2).

4 The `memvarTimed<T, Time, Clock>` Class

4.1 Purpose and template parameters

`memvarTimed` is a `final` class that publicly derives from `memvar<T, P>`, where `P` is `detail::TimeTagPolicy<Time,Clock>`. It stores, for every value in the history, the time at which the value was written. The two extra template parameters default to `std::chrono::nanoseconds` (`Time`, the resolution of the tags) and `std::chrono::steady_clock` (`Clock`, the time source).

Listing 10: The time-tagged variable (memvar.h).

```
437 // memvar whose every value also carries a time tag. Each history entry holds
438 // the value and its time point together (one deque), so the two can never get
439 // out of step, whether a write succeeds or throws.
440 template <MemVarType T,
441           typename Time = std::chrono::nanoseconds,
442           typename Clock = std::chrono::steady_clock>
443 class memvarTimed final : public memvar<T, detail::TimeTagPolicy<Time, Clock>>
        {
444   using MemvarBase = memvar<T, detail::TimeTagPolicy<Time, Clock>>;
445 public:
446   using capacityType = typename MemvarBase::capacityType;
447   // a T value, the time tag, and an error flag (false: no error)
448   using historyTimedValue = std::tuple<T, Time, bool>;
449
450   memvarTimed() : memvarTimed(T{}) {}
451
452   explicit memvarTimed(const T& value, const capacityType historyCapacity =
          MemvarBase::historyCapacityDefault_) :
453     MemvarBase(value, historyCapacity), memvarEpoch_(this->memo_.front().tag)
          {}
454
455   memvarTimed(const memvarTimed&) = delete;
456   memvarTimed(memvarTimed&&) = delete;
457   memvarTimed& operator=(memvarTimed&&) = delete;
458
459   // Not inherited from MemvarBase on purpose: MemvarBase::operator= would
          return a MemvarBase&.
460   memvarTimed& operator=(const T& rhs) { this->setValue(rhs); return *this; }
461   memvarTimed& operator=(T&& rhs) { this->setValue(std::move(rhs)); return *
          this; }
462   memvarTimed& operator=(const memvarTimed& rhs) { this->setValue(rhs.getValue
          ()); return *this; }
463
464   // time elapsed from the memvar's epoch (the time of its first value) to the
465   // write of the i-th value
466   Time getTimeTag(const capacityType index = 0) const {
467     return std::chrono::duration_cast<Time>(this->slot(index).tag -
          memvarEpoch_);
468   }
469
470   historyTimedValue getHistoryTimedValue(const capacityType index) const {
471     const auto& s {this->slot(index)};
```

```
472     return std::make_tuple(s.value, std::chrono::duration_cast<Time>(s.tag -
            memvarEpoch_), false);
473   }
474
475   void printHistoryTimedData(std::ostream& os = std::cout, const std::string&
          separator = "\n") const {
476     printTimed(os, separator, false);
477   }
478   void printReverseHistoryTimedData(std::ostream& os = std::cout, const std::
          string& separator = "\n") const {
479     printTimed(os, separator, true);
480   }
481
482   void clearHistory() override {
483     MemvarBase::clearHistory();                  // single new entry, tagged now
484     memvarEpoch_ = this->memo_.front().tag;      // and it is the new epoch
485   }
486
487 private:
488   void printTimed(std::ostream& os, const std::string& separator, const bool
          reverse) const {
489     const std::size_t n {this->memo_.size()};
490     os << "{ --- begin ---\n[TimeTag:Value]\n";
491     for ( std::size_t k {0}; k < n; ++k ) {
492       const auto& s {this->memo_[reverse ? n - 1 - k : k]};
493       os << "[" << std::chrono::duration_cast<Time>(s.tag - memvarEpoch_).
              count()
494          << ":" << s.value << "]" << separator;
495     }
496     os << "  --- end --- }\n\n";
497   }
498
499   typename detail::TimeTagPolicy<Time, Clock>::tag_type memvarEpoch_;
500 };  // class memvarTimed
```

4.2 State and invariants

- The history is the base class's `std::deque<Stamped<T, tag>>`: each element is one value together with the `time_point` at which it was written. There is no parallel container, so the invariant "one tag per value" holds by construction and needs no maintenance.
- `memvarEpoch_` – the only member added by `memvarTimed`. The constructor sets it to the tag of the first entry, which the base constructor has just stored. Reported tags are offsets from the epoch, so the first value has the tag zero *exactly* (not merely approximately).
- Time tags are stored with the resolution `Time` (the policy truncates the clock reading with `time_point_cast`); the derived accessors apply `duration_cast<Time>` to the difference between a tag and the epoch.

4.3 The write path

There is no hook. A timed write is the base class's `setValue`, whose `pushFront` emplaces the value together with `Policy::now()`: one clock read and one insertion (about 24 ns on the test machine, Table 8, nearly all of it the clock call). Because insertion is the only step that can throw (Section 3.4), a timed write has the strong exception guarantee, and the value and tag of every entry are inserted and evicted as one unit.

4.4 Reading time tags

- `getTimeTag(index = 0)` returns the duration from the epoch to the write of the entry, `duration_cast` to `Time`. The parameter is the signed `capacityType` and the entry is fetched with the same bounds-checked `slot(i)` as the value accessors: a negative or too large index throws `std::out_of_range` with the identical message `Index` *i* `is out of bounds (size is` *n*`)`.
- `getHistoryTimedValue(index)` returns `std::tuple<T, Time, bool>` built from one entry (one lookup, one check).
- `printHistoryTimedData` and `printReverseHistoryTimedData` share the private `printTimed`: a header (`{ -- begin --`, a column title `[TimeTag:Value]`, one `[tag:value]` item per entry followed by the separator, and the footer `-- end -- }` plus a blank line). The tag is printed with `.count()`, i.e. without its unit.
- Streaming a `memvarTimed` with `operator«` prints `[tag-of-newest:current-value]`.

4.5 Overrides and name hiding

- `clearHistory()` (override) calls the base version, which inserts one stamped `T{}` and erases the rest, and then makes that entry's tag the new epoch: after a clear the single remaining value has tag exactly zero relative to the *new* epoch, so tags are not comparable across a clear. If the base version throws, the epoch is not touched and the history is unchanged.
- The three assignment operators are redeclared so that they return `memvarTimed&`; as a side effect they *hide* the base-class assignments. Assigning a bare `memvar<T>` to a `memvarTimed` therefore works only through the implicit `operator T()` followed by `operator=(T&&)` (one extra copy of `T`); this is exercised by the unit tests.
- Everything else (`operator()`, `operator[]`, `++`, `-`, the history getters, `cref()`, printing of values) is inherited. The compound assignments are free templates, so they apply to the derived class without being redeclared.
- Copy/move construction and move assignment are deleted.

4.6 What the single-deque design changed

The earlier versions of the header (rel.1.2.x or older) stored a second `std::deque` of time points next to the value deque. A failure of the construction (for example `std::bad_alloc` while copying a large string) therefore left the two deques with different sizes, and every later pairing of value and tag was wrong (Section 8.5 reproduces this against the old header and shows that the new one is unaffected). The present design removes the cause rather than patching the symptom. Table 6 lists the measured and structural differences.

The last four rows are the price: the type relationship and the shape of the history container changed, which affects code that was written against the old interface (issue I17). Run-time cost is unchanged within measurement noise.

4.7 Choice of Time and Clock

The policy's `now()` is `time_point_cast<Time>(Clock::now())`. Consequently

- a coarser `Time` (milliseconds) truncates; a finer one (picoseconds) is allowed and merely scaled;
- a floating-point duration such as `duration<double>` works and its `.count()` is a `double` (verified by the example in Section 7);

Property	Former 1.2.x (two deques)	Current 1.3.0 (one deque)
Failed write on `memvarTimed`	desynchronises value and tags	no effect (strong guarantee)
`sizeof(memvarTimed<int>)`, mode 2 / modes 0,1	184 / 192	104 / 112
Heap allocations to construct `memvarTimed<int>(0,10)`	5 (1 664 B)	3 (1 088 B)
Virtual functions	dtor, `clearHistory`	dtor, `clearHistory`
Timed write, ns (same machine and build)	23.8	24.0
Plain write, ns (same machine and build)	2.36	2.40
`memvarTimed<int>` converts to `const memvar<int>&`	yes	**no**
Type returned by `getMemVarHistory()` of a timed variable	`deque<T>&`	`deque<Stamped<T,tag>>&`
Index-error message of `getTimeTag`	standard library's text	same as the value accessors
Tag of the first value	zero (epoch taken afterwards)	zero exactly (epoch := its tag)

Table 6: Former and current design of `memvarTimed` (GCC 13.3, libstdc++, x86-64; the two benchmark rows are the best of three interleaved runs of the benchmark of Table 8, mode 2).

- a non-monotonic clock such as `system_clock` may produce tags that decrease with a clock adjustment, since nothing in the class enforces monotonicity.

5 Free Functions and Namespace-Scope Operator Overloads

5.1 Why a friend struct and a concept

The operators are *not* members, so that the same code serves `memvar` and `memvarTimed`, and so that both operand orders (`m + 1` and `1 + m`) are covered by one template. They need the protected getter and setter, which is granted to a single helper:

Listing 11: Operator plumbing (memvar.h).

```
204 // Gives the free operators below access to memvar's protected value/setter.
205 struct memvarAccess {
206   template <MemVarType T, typename P>
207   static const T& get(const memvar<T, P>& m) { return m.getValue(); }
208
209   template <MemVarType T, typename P, typename U>
210   static void set(memvar<T, P>& m, U&& v) { m.setValue(std::forward<U>(v)); }
211 };
212
213 template <typename X>
214 concept IsMemvar = requires(const X& x) { memvarAccess::get(x); };
215
216 // The current value if x is a memvar (no copy), otherwise x itself.
217 template <typename X>
218 constexpr const auto& valueOf(const X& x) {
219   if constexpr ( IsMemvar<X> ) return memvarAccess::get(x);
220   else return x;
221 }
222
223 // "L Op R is valid on the underlying values and at least one side is a memvar
    ".
```

```
224 template <typename Op, typename L, typename R>
225 concept MemvarOperands =
226   (IsMemvar<L> || IsMemvar<R>) &&
227   std::is_invocable_v<Op, decltype(valueOf(std::declval<const L&>())),
228                           decltype(valueOf(std::declval<const R&>()))>;
```

- `memvarAccess` is a `friend` of `memvar<T, P>`. Its `get` takes `const memvar<T, P>&`; template deduction from a `memvarTimed<...>` argument succeeds by derived-to-base deduction (`P` is deduced as the time-tag policy), which is how one set of operators serves both flavours although they are unrelated types from the point of view of non-template code.
- `IsMemvar<X>` is true exactly when `memvarAccess::get` can be called with an `X`; for any type that is not a memvar, deduction fails and the concept is false.
- `valueOf(x)` returns `const auto&`: the stored current value when `x` is a memvar (no copy), otherwise `x` itself.
- `MemvarOperands<Op, L, R>` states that at least one operand is a memvar and that the standard function object `Op` (`std::plus<>`, `std::less<>`, ...) is invocable on the underlying values. The concept is a conjunction whose first operand is false for two non-memvar types, so the templates are inert for all unrelated expressions.

5.2 The generated operators

Listing 12: Binary, comparison and compound-assignment operators (memvar.h).

```
512 // Binary operators between a memvar (bare or timed) and another memvar or a
513 // plain value, on either side. One constrained template per operator replaces
514 // the former three overloads each; operands are read by const reference.
515 #define MEMVAR_BINARY_OPERATOR(OP, FUNCTOR)
                                                \
516   template <typename L, typename R>
                                                      \
517     requires MemvarOperands<FUNCTOR, L, R>
                                                   \
518   auto operator OP(const L& lhs, const R& rhs) {
                                                  \
519     return valueOf(lhs) OP valueOf(rhs);
                                                        \
520   }
521
522 MEMVAR_BINARY_OPERATOR(==, std::equal_to<>)
523 MEMVAR_BINARY_OPERATOR(<,  std::less<>)
524 MEMVAR_BINARY_OPERATOR(>,  std::greater<>)
525 MEMVAR_BINARY_OPERATOR(<=, std::less_equal<>)
526 MEMVAR_BINARY_OPERATOR(>=, std::greater_equal<>)
527 MEMVAR_BINARY_OPERATOR(+,  std::plus<>)
528 MEMVAR_BINARY_OPERATOR(-,  std::minus<>)
529 MEMVAR_BINARY_OPERATOR(*,  std::multiplies<>)
530 MEMVAR_BINARY_OPERATOR(/,  std::divides<>)
531 #undef MEMVAR_BINARY_OPERATOR
532
533 // Compound assignment: the left side must be a memvar; it records a new value
    .
534 #define MEMVAR_COMPOUND_OPERATOR(OP, FUNCTOR)
                                                \
535   template <typename L, typename R>
                                                      \
536     requires (IsMemvar<L> && MemvarOperands<FUNCTOR, L, R>)
                                    \
```

```
537   L& operator OP##=(L& lhs, const R& rhs) {
                                                \
538     memvarAccess::set(lhs, valueOf(lhs) OP valueOf(rhs));
                                    \
539     return lhs;
                                                                    \
540   }
541
542 MEMVAR_COMPOUND_OPERATOR(+, std::plus<>)
543 MEMVAR_COMPOUND_OPERATOR(-, std::minus<>)
544 MEMVAR_COMPOUND_OPERATOR(*, std::multiplies<>)
545 MEMVAR_COMPOUND_OPERATOR(/, std::divides<>)
546 #undef MEMVAR_COMPOUND_OPERATOR
```

Two macros stamp out one function template per operator (they are `#undef`ed afterwards):

Operators	**Signature (conceptually)**	**Behaviour**
`== < > <= >=`	`bool op(const L&, const R&)`	compare the underlying values; at least one side is a memvar or memvarTimed, the other any type comparable with `T`
`+ - * /`	`auto op(const L&, const R&)`	return a *plain value* of the type of the underlying expression, never a memvar
`+= -= *= /=`	`L& op(L&, const R&)`	`L` must be a memvar; computes the new value, then records it with `memvarAccess::set`; returns the left operand
`!=`	(synthesised)	C++20 rewrites `a != b` as `!(a == b)`; reversed forms of == are also synthesised

Table 7: Operator families.

Observations:

- Operands may be `memvar`, `memvarTimed`, `T`, or any type for which the underlying operation compiles: `memvar<string> == "abc"`, `"<" + s`, `memvar<bigint> + 7`, and comparisons between different memvar value types such as `memvar<int> == memvar<long>`.
- Because the result is `auto` of the underlying expression, it is *not* re-narrowed to `T`: `memvar<int8_t> + memvar<int8_t>` is an `int`, `memvar<int> + 1.5` is a `double` (`static_asserted` in the edge-case program).
- Compound assignment writes `valueOf(lhs) OP valueOf(rhs)` into the history: if the type of that result differs from `T` it is converted by `emplace_front`'s direct-initialisation, *silently truncating* (`memvar<int> += 1.9` records 2; `memvar<int8_t>(120) += 20` records −116).
- The operators for `-`, `*`, `/` and their compound forms are absent for strings because `std::minus<>` and friends are not invocable on strings; no explicit `requires` is needed.
- The body is now `valueOf(lhs) OP valueOf(rhs)` written directly, not through the standard function object, so a compiler diagnostic about the underlying operation (for instance `-Wsign-compare` for mixed signedness) is reported at the macro line inside `memvar.h` with the user's expression only as "required from here" (Section 8).
- There are no `%`, `%=`, unary `-`, bitwise or shift operators. For arithmetic `T` some of them still compile through the implicit `operator T()` and return a plain value (`m % 3`); for `bigint`, whose `%` and unary minus are *member* operators, they do not (issue I6).
- The templates live in namespace `mv` and are found by argument dependent lookup.

5.3 Stream output

Listing 13: Stream operators for memvar and memvarTimed (memvar.h).

```
550 template <typename T, typename P>
551 std::ostream& operator<<(std::ostream& os, const mv::memvar<T, P>& mv) {
552   return os << mv::valueOf(mv);
553 }
554
555 template <typename T, typename Time, typename Clock>
556 std::ostream& operator<<(std::ostream& os, const mv::memvarTimed<T, Time,
        Clock>& mvt) {
557   return os << "[" << mvt.getTimeTag().count() << ":" << mv::valueOf(mvt) << "
        ]";
558 }
```

The two `operator«` templates are declared in the *global* namespace; the first is a template on `<T, P>`, the second is the more specialised overload for `memvarTimed<T, Time, Clock>` (an exact match, which beats the derived-to-base conversion needed for the first). A `memvar<T>` prints as its current value; a `memvarTimed` prints as `[tag:value]`. Because they are in the global namespace they are not found by argument-dependent lookup (the associated namespace of `memvar<T>` is `mv`), only by ordinary lookup, and ordinary lookup stops at the first enclosing scope that declares any `operator«`. Section 8 shows the consequence.

5.4 Unicode output helpers

Listing 14: UTF-8 conversion helpers (memvar.h).

```
81  template <typename CharT>
82  std::string toUtf8(const std::basic_string<CharT>& s) {
83    using Unit = std::make_unsigned_t<CharT>;
84    std::string out;
85    out.reserve(s.size());
86    for ( std::size_t i {0}; i < s.size(); ) {
87      char32_t cp {static_cast<Unit>(s[i++])};
88      if constexpr ( sizeof(CharT) == 2 ) {
89        if ( cp >= 0xD800 && cp <= 0xDBFF && i < s.size() ) {
90          const char32_t low {static_cast<Unit>(s[i])};
91          if ( low >= 0xDC00 && low <= 0xDFFF ) {
92            cp = 0x10000 + ((cp - 0xD800) << 10) + (low - 0xDC00);
93            ++i;
94          }
95        }
96      }
97      appendUtf8(out, cp);
98    }
99    return out;
100 }
101
102 [[noreturn]] inline void throwIndexOutOfRange(const int64_t index, const std::
        size_t size) {
103   throw std::out_of_range("Index " + std::to_string(index) +
104                           " is out of bounds (size is " + std::to_string(size)
                                + ").");
105 }
106 }  // namespace mv::detail
107
108 // Stream output for the non-char standard strings (UTF-8 on the wire).
109 template <typename CharT>
110   requires (std::same_as<CharT, wchar_t> || std::same_as<CharT, char16_t> ||
        std::same_as<CharT, char32_t>)
```

```
111   std::ostream& operator<<(std::ostream& os, const std::basic_string<CharT>& s
          ) {
112     return os << mv::detail::toUtf8(s);
113 }
114
115 inline std::ostream& operator<<(std::ostream& os, const std::u8string& s) {
116   return os << std::string_view(reinterpret_cast<const char*>(s.data()), s.
          size());
117 }
```

The standard library has no `ostream` insertion for `wstring`, `u16string`, `u32string` or `u8string`, and C++20 deleted the overloads for the corresponding character pointers. The header supplies them, in the global namespace, so that `memvar<std::wstring>` and the others can be printed (and so that the values in a history can be dumped):

- `appendUtf8` encodes one code point into 1–4 bytes; surrogates and values above `U+10FFFF` become `U+FFFD`.
- `toUtf8<CharT>` reads each code unit through `std::make_unsigned_t<CharT>`, and, for 2-byte code units (`char16_t` and `wchar_t` where it is 16 bits wide), combines a high surrogate followed by a low surrogate into one code point. A lone surrogate (including a high surrogate at the very end) is replaced by `U+FFFD`.
- the `requires` clause restricts the template to `wchar_t`, `char16_t` and `char32_t`; `u8string` is forwarded byte for byte through a `string_view`.

5.5 Other free functions

`mv::getHistoryValue(const memvar<T, P>&, i)` and `mv::getHistoryTimedValue(const memvarTimed<...>&, i)` both return `T` and are implemented as `mv(i)`. The latter therefore *drops the time tag* and has a different return type than the member of the same name (issue I9).

6 Compilation and Compilation Flags

6.1 Language and library requirements

The header needs a C++20 compiler and standard library. The features it relies on are concepts and constrained templates (`concept`, `requires`, trailing `requires` on non-template members), `std::same_as`, `std::remove_cvref_t`, `std::u8string`, the `[[unlikely]]` attribute, `noexcept(constant-expression)`, `if constexpr`, and `std::ranges::minmax_element`. The code was built and run with g++ 13.3 and libstdc++ (the project's CMake file selects g++ and notes that it performed better than `clang++` on the authors' machine, an Ubuntu 24.04.5). The unit-test file additionally replaces the global `operator new/delete` and silences GCC's `-Wmismatched-new-delete` for that pair (Section 9); that diagnostic was reported by GCC 16.2.1 and reproduced with g++ 13 and 14 at `-O2/-O3`. GCC 16 itself and `clang++` were not available for this report.

6.2 Build configuration of the project

The unit-test project uses CMake and sets the following (abridged):

Listing 15: Relevant parts of the project's CMakeLists.txt.

```
CMAKE_MINIMUM_REQUIRED(VERSION 3.26)
SET(CMAKE_C_COMPILER gcc)
SET(CMAKE_CXX_COMPILER g++)
if(NOT DEFINED MEMVAR_SAFEREF_CHECK)
```

```
    SET(MEMVAR_SAFEREF_CHECK 2)          # default: MEMVAR_SAFEREF_NO_CHECK
endif()
SET(CMAKE_CXX_STANDARD 20)
SET(CMAKE_CXX_STANDARD_REQUIRED ON)
SET(CMAKE_CXX_EXTENSIONS OFF)
SET(CMAKE_BUILD_TYPE Release)
SET(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra -pedantic -march=native
    ")   # in the Release branch
FIND_PACKAGE(Threads REQUIRED)
FIND_PACKAGE(GTest REQUIRED)
SET(SOURCE_FILES unitTests.cpp bigint.cpp)
TARGET_COMPILE_DEFINITIONS(${THE_PROJECT} PRIVATE MEMVAR_SAFEREF_CHECK=${
    MEMVAR_SAFEREF_CHECK})
TARGET_LINK_LIBRARIES(${THE_PROJECT} PRIVATE Threads::Threads ${
    GTEST_LIBRARIES} ${GTEST_MAIN_LIBRARIES})
```

Two details of this file are worth knowing: `CMAKE_BUILD_TYPE` is set unconditionally to `Release`, so a `-DCMAKE_BUILD_TYPE=Debug` given on the command line is overridden and the Debug branch (extra warning flags) is never taken; and the macro value is passed on without validation (an invalid value is stopped later by the header's `#error`). GoogleMock headers are included by the tests, but only the GTest libraries are linked, which suffices because no GoogleMock symbol that needs a library is used.

With the Release build type CMake adds `-O3 -DNDEBUG`. The header itself does not use `assert`, so `NDEBUG` has no effect on it. The unit tests are built with:

Listing 16: Building and running the tests in each safety mode.

```
cmake -DMEMVAR_SAFEREF_CHECK=0 -S . -B build && cmake --build build   # abort
    mode
cmake -DMEMVAR_SAFEREF_CHECK=1 -S . -B build && cmake --build build   #
    throwing mode
cmake -DMEMVAR_SAFEREF_CHECK=2 -S . -B build && cmake --build build   #
    default, no check
./build/memvar-unit-tests --gtest_filter=-memVarTest.test_6
```

(`test_6` constructs a history of one billion entries and was excluded on the memory-limited machine used here.)

6.3 Rules that follow from the code

- **Include order for `bigint`.** `memvar.h` only forward declares `bip::bigint`. Printing a `memvar<bip::bigint>` calls `operator«(ostream&, const bip::bigint&)`, which is declared in the *global* namespace in `bigint.h`; it is found only if `bigint.h` is included *before* `memvar.h` (ADL looks in `bip`, not in the global namespace).
- **One value of `MEMVAR_SAFEREF_CHECK` per program.** The macro changes the class layout: the member `writeCount_` and the nested class `SafeRef` exist only in the checked modes, and `sizeof(memvar<int>)` is 96 bytes in mode 2 and 104 bytes in modes 0 and 1 (112 versus 104 for `memvarTimed<int>`). Translation units compiled with different values violate the one-definition rule (issue I3).
- **Inline everything.** All functions are templates or `inline`, so no library needs to be linked for `memvar.h` itself (the project links `bigint.cpp` for `bigint`).

6.4 Run-time cost

A write costs about the same as a bare `deque` `push_front` plus `pop_back`, i.e. the wrapper adds essentially nothing over the deque it wraps, and the safety modes are indistinguishable from the default

Operation (best of 7, ns per operation)	mode 2	mode 0	mode 1
raw `long long` write (reference)	0.22	0.22	0.22
`memvar<long long>` write, capacity 10	2.44	2.46	2.51
alternating read and write	2.68	2.79	2.79
`m += 1`	2.58	2.51	2.52
`memvarTimed<long long>` write	23.84	24.25	23.97
hand-written deque `push_front`/`pop_back`	2.45	2.58	2.47

Table 8: Micro-benchmark of the delivered header (GCC 13.3, `-O3 -march=native`, 20 M iterations, 5 M for the timed write; virtual machine; differences below about 0.2 ns are within noise).

within the measurement noise. The timed variant is dominated by the clock read. The same benchmark compiled against the former header gives the same figures within noise (Table 6); the benchmark does not, however, exercise the write path through a base-class reference, where the former virtual call could not always be removed.

Quantity (libstdc++, x86-64)	mode 2	modes 0/1
`sizeof(memvar<int>)` (same for `double`, `std::string`)	96	104
`sizeof(memvarTimed<int>)`	104	112
`sizeof(Stamped<int, steady_clock::time_point>)`	16	16
`sizeof(std::deque<int>)`	80	80
heap allocations to construct `memvar<int>(0, 10)` (or capacity 1000)	3 (1088 B)	3 (1088 B)
heap allocations to construct `memvarTimed<int>(0, 10)`	3 (1088 B)	3 (1088 B)
heap allocations for 10^6 writes, `memvar<int>`, capacity 10	7812	7812
heap allocations for 10^6 writes, `memvar<long long>`, capacity 10	15625	15625
heap allocations for 10^6 writes, `memvarTimed<int>`, capacity 10	31250	31250

Table 9: Memory footprint and allocation behaviour (modes 0/1 columns: the size rows were measured in mode 0; the allocation rows are independent of the mode).

7 Complete Usage Examples

All programs in this section are complete (they compile as shown with `g++ -std=c++20 -I. file.cpp`, adding `bigint.cpp` where `bigint.h` is used), were compiled with `-Wall -Wextra -pedantic` without warnings, and the output printed below each one is the output of that run. Each uses the default safety mode unless stated.

7.1 Basic use: history, indexing, printing, clearing

Listing 17: Basic use of memvar<int>.

```
#include <iostream>
#include "memvar.h"

int main() {
  mv::memvar<int> counter(0, 4);          // initial value 0, history capacity 4

  for (int i = 1; i <= 6; ++i) {
    counter = i * 10;                      // every assignment is recorded
  }

  std::cout << "current      : " << counter << '\n';
  std::cout << "size/capacity: " << counter.getHistorySize() << '/'
```

```
            << counter.getHistoryCapacity() << '\n';
  std::cout << "full         : " << std::boolalpha << counter.isHistoryFull()
      << '\n';

  // index 0 is the newest value, index size-1 the oldest one still stored
  std::cout << "counter(0)=" << counter(0) << " counter[1]=" << counter[1]
            << " valueAt(3)=" << counter.valueAt(3) << '\n';

  counter.printHistoryData();            // newest -> oldest
  counter.printReverseHistoryData(std::cout, ", ");   // oldest -> newest

  const auto [mn, mx] = counter.getHistoryMinMax();
  std::cout << "min=" << mn << " max=" << mx << '\n';

  const auto [value, error] = counter.getHistoryValue(2);
  std::cout << "getHistoryValue(2) -> (" << value << ", " << error << ")\n";

  // implicit conversion to T, and the call operator without arguments
  int plain = counter;
  std::cout << "plain=" << plain << " counter()=" << counter() << '\n';

  counter.clearHistory();                // history shrinks to one value: T{}
  std::cout << "after clear: " << counter << " size=" << counter.
      getHistorySize() << '\n';

  // the free function form
  counter = 7;
  counter = 8;
  std::cout << "free getHistoryValue(counter, 1) = " << mv::getHistoryValue(
      counter, 1) << '\n';
}
```

Listing 18: Output of Listing 17.

```
current      : 60
size/capacity: 4/4
full         : true
counter(0)=60 counter[1]=50 valueAt(3)=30
[ 60 50 40 30 ]
[ 30, 40, 50, 60, ]
min=30 max=60
getHistoryValue(2) -> (40, false)
plain=60 counter()=60
after clear: 0 size=1
free getHistoryValue(counter, 1) = 7
```

Points to note: the initial value 0 and the writes 10 and 20 were evicted because the capacity is 4 (so `size/capacity` is `4/4` and `isHistoryFull()` is true); `printReverseHistoryData` lists oldest to newest and also leaves a trailing separator; `clearHistory()` resets the current value to 0, not to the last value; after clearing, the two following writes give a history in which index 1 is the value 7.

7.2 Operators: arithmetic, comparison, compound assignment

Listing 19: Arithmetic, comparison, compound assignment, increments and strings.

```
#include <iostream>
#include <string>
#include "memvar.h"

int main() {
  mv::memvar<int> a(5), b(7);
```

```
7
8     std::cout << "a+b=" << a + b << " a*3=" << a * 3 << " 100-a=" << 100 - a
9               << " b/2=" << b / 2 << '\n';
10    std::cout << std::boolalpha
11              << "a<b: " << (a < b) << "  a==5: " << (a == 5)
12              << "  7==b: " << (7 == b) << "  a!=b: " << (a != b) << '\n';
13
14    a += 10;          // records 15
15    a -= b;           // records 8   (memvar operand)
16    a *= 2;           // records 16
17    a /= 4;           // records 4
18    std::cout << "a after compound ops: " << a << '\n';
19    a.printHistoryData();
20
21    // prefix returns the new value, postfix the old one; both record a new
          value
22    std::cout << "++a=" << ++a << " a++=" << a++ << " now a=" << a << '\n';
23    std::cout << "--a=" << --a << " a--=" << a-- << " now a=" << a << '\n';
24
25    // the result of a binary operator is a plain value, not a memvar
26    auto sum = a + b;
27    static_assert(std::is_same_v<decltype(sum), int>);
28
29    // statements that read and write the same variable are fine
30    a = a + a + b;
31    std::cout << "a = a + a + b -> " << a << '\n';
32
33    // strings: only + and += (and the comparisons) are available
34    mv::memvar<std::string> s("foo", 3);
35    s += "bar";
36    s = s + "!" ;
37    s = "<" + s + ">";
38    std::cout << s << " size=" << s.getHistorySize() << '\n';
39    s.printHistoryData(std::cout, " | ");
40    std::cout << "s == \"<foobar!>\": " << (s == "<foobar!>") << '\n';
41
42    // floating point
43    mv::memvar<double> d(1.5);
44    d *= 2.0;
45    d = d / 4.0;
46    std::cout << "d=" << d << " history: ";
47    d.printHistoryData();
48 }
```

Listing 20: Output of Listing 19.

```
a+b=12 a*3=15 100-a=95 b/2=3
a<b: true  a==5: true  7==b: true  a!=b: true
a after compound ops: 4
[ 4 16 8 15 5 ]
++a=5 a++=5 now a=6
--a=5 a--=5 now a=4
a = a + a + b -> 15
<foobar!> size=3
[ <foobar!> | foobar! | foobar | ]
s == "<foobar!>": true
d=0.75 history: [ 0.75 3 1.5 ]
```

The compound operators each record a new value, which is why the history after four of them shows five entries (`[ 4 16 8 15 5 ]`). The expression `a = a + a + b` reads the current value twice and records the result once. The chain `"++a=" « ++a « " a++=" « a++` is well defined because C++17 sequences

the operands of « left to right. The `static_assert` shows that `a + b` has type `int`, not `memvar<int>`.

7.3 Time-tagged variables

Listing 21: memvarTimed with default and custom Time/Clock.

```cpp
#include <chrono>
#include <iostream>
#include <sstream>
#include <thread>
#include "memvar.h"

int main() {
  using namespace std::chrono_literals;

  // default Time = nanoseconds, Clock = steady_clock
  mv::memvarTimed<int> t(100, 3);
  for (int i = 1; i <= 4; ++i) {
    std::this_thread::sleep_for(5ms);
    t = 100 + i;
  }

  std::cout << "value history (newest first): ";
  t.printHistoryData();

  // time tags are offsets from the memvar's own epoch, newest first
  bool monotonic = true;
  for (std::size_t i = 1; i < static_cast<std::size_t>(t.getHistorySize()); ++
      i) {
    monotonic = monotonic && (t.getTimeTag(i - 1) > t.getTimeTag(i));
  }
  std::cout << "tags strictly decreasing with index: " << std::boolalpha <<
      monotonic << '\n';
  std::cout << "newest tag >= 20 ms: " << (t.getTimeTag() >= 20ms) << '\n';

  const auto [v, tag, err] = t.getHistoryTimedValue(1);
  std::cout << "getHistoryTimedValue(1): value=" << v
            << " tag>0: " << (tag.count() > 0) << " err=" << err << '\n';

  // output format of the full timed dump (tags vary from run to run)
  std::ostringstream os;
  t.printHistoryTimedData(os, ";");
  const std::string dump = os.str();
  std::cout << "dump has header: " << (dump.rfind("{ --- begin ---\n[TimeTag:
      Value]\n", 0) == 0)
            << ", footer: " << dump.ends_with("  --- end --- }\n\n") << '\n';

  // custom resolution and clock
  mv::memvarTimed<double, std::chrono::milliseconds, std::chrono::system_clock
      > w(1.0, 5);
  std::this_thread::sleep_for(12ms);
  w = 2.0;
  std::cout << "ms tag >= 12ms: " << (w.getTimeTag() >= 12ms) << '\n';

  // floating-point duration
  using fsec = std::chrono::duration<double>;
  mv::memvarTimed<int, fsec> f(1);
  f = 2;
  std::cout << "fsec tag is double: "
            << std::is_same_v<decltype(f.getTimeTag().count()), double> << '\n
                ';

  // clearHistory() restarts the epoch: the single remaining value has tag 0
```

```
  t.clearHistory();
  std::cout << "after clear: value=" << t << " size=" << t.getHistorySize() <<
      '\n';

  // streaming a timed memvar prints "[newest tag:value]"
  std::ostringstream one;
  one << t;
  std::cout << "streamed form begins with '[': " << (one.str().front() == '[')
      << '\n';
}
```

Listing 22: Output of Listing 21.

```
value history (newest first): [ 104 103 102 ]
tags strictly decreasing with index: true
newest tag >= 20 ms: true
getHistoryTimedValue(1): value=103 tag>0: true err=false
dump has header: true, footer: true
ms tag >= 12ms: true
fsec tag is double: true
after clear: value=[0:0] size=1
streamed form begins with '[': true
```

The actual tag values depend on the run, so the program checks properties instead of printing them: tags decrease with the index, the newest tag is at least the sum of the sleeps, and the dump has the documented header and footer. The third object uses `duration<double>` as `Time`. The last lines show that `clearHistory()` leaves one value with tag zero, and that streaming a timed variable prints the tag/value pair, here `[0:0]`.

7.4 Big integers: a Fibonacci recurrence

Listing 23: memvar<bip::bigint> with bigint.h included first.

```
#include <iostream>
#include "bigint.h"     // must come BEFORE memvar.h (see Section on
    constraints)
#include "memvar.h"

int main() {
  // Fibonacci with a memory of the previous two terms only
  mv::memvar<bip::bigint> fib(bip::bigint(1), 2);
  fib = bip::bigint(1);                          // fib(0) = 1, fib(1) = 1
  for (int i = 3; i <= 100; ++i) {
    fib += fib(1);                               // new = current + previous
  }
  std::cout << "F(100) = " << fib << '\n';

  // mixed bigint / int32_t arithmetic is forwarded to bigint's own operators
  mv::memvar<bip::bigint> big(bip::bigint("10000000000000000000"), 4);
  big = big * 3;
  big += 7;
  std::cout << "big = " << big << '\n';
  std::cout << "big > 5: " << std::boolalpha << (big > 5) << '\n';
  std::cout << "history size = " << big.getHistorySize() << '\n';

  try {
    big /= bip::bigint(0);                       // bigint throws; nothing is
        recorded
  } catch (const std::invalid_argument& e) {
    std::cout << "caught: " << e.what() << ", history size still "
              << big.getHistorySize() << '\n';
```

```
  }
}
```

Listing 24: Output of Listing 23.

```
F(100) = 354224848179261915075
big = 300000000000000000007
big > 5: true
history size = 3
caught: Division by zero, history size still 3
```

A capacity of 2 is the minimum and is exactly what a two-term recurrence needs: after `fib = 1` the history is `[1, 1]`, and `fib += fib(1)` computes "current plus previous" and records it, which evicts the older of the two. The failed division demonstrates that the arithmetic in a compound assignment is evaluated *before* anything is recorded: the history size is still 3 after the exception.

7.5 A realistic pattern: a sensor filter

Listing 25: Moving average and jump detection on top of getMemVarHistory().

```
#include <iostream>
#include <numeric>
#include "memvar.h"

// A small sensor filter: keep the last 5 samples, expose a moving average
// and a "changed more than a threshold since the previous sample" test.
class Sensor {
public:
  void sample(int raw) { raw_ = raw; }
  double movingAverage() const {
    const auto& h = raw_.getMemVarHistory();            // const std::deque<
        int>&
    return std::accumulate(h.begin(), h.end(), 0.0) / static_cast<double>(h.
        size());
  }
  bool jumped(int threshold) const {
    if (raw_.getHistorySize() < 2) return false;
    const int delta = raw_(0) - raw_(1);                // newest - previous
    return (delta < 0 ? -delta : delta) > threshold;
  }
  int last() const { return raw_(); }
private:
  mv::memvar<int> raw_ {0, 5};
};

int main() {
  Sensor s;
  for (int v : {10, 11, 9, 10, 40, 41, 42}) {
    s.sample(v);
    std::cout << "sample " << v << "  avg(last 5) = " << s.movingAverage()
              << (s.jumped(15) ? "   <-- jump" : "") << '\n';
  }
}
```

Listing 26: Output of Listing 25.

```
sample 10  avg(last 5) = 5
sample 11  avg(last 5) = 7
sample 9  avg(last 5) = 7.5
sample 10  avg(last 5) = 8
```

```
sample 40  avg(last 5) = 16   <-- jump
sample 41  avg(last 5) = 22.2
sample 42  avg(last 5) = 28.4
```

This is the intended style of use: the owner of the variable assigns raw samples, and the history provides both the moving window (iterated without copying through `getMemVarHistory()`) and the previous sample (`raw_(1)`). The first averages are low because the window initially holds the constructor's `0`.

7.6 Dangling-reference protection in the three modes

Listing 27: Using cref() and observing the effect of MEMVAR_SAFEREF_CHECK.

```
#include <iostream>
#include "memvar.h"

int main() {
  mv::memvar<int> m(1, 3);                          // capacity 3
  auto h = m.cref();                                // handle on the value 1
  m = 2;
  m = 3;
  std::cout << "after 2 more writes: " << h << '\n';   // still alive (3
      values stored)

#if MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_THROWS
  m = 4;                                            // evicts the slot h refers
      to
  try {
    std::cout << h << '\n';
  } catch (const mv::expired_history_reference& e) {
    std::cout << "expired handle detected: " << e.what() << '\n';
  }
#elif MEMVAR_SAFEREF_CHECK == MEMVAR_SAFEREF_ABORT
  m = 4;                                            // evicts the slot h refers
      to
  std::cout << "dereferencing the expired handle now aborts the process\n";
  std::cout.flush();
  std::cout << h << '\n';                           // prints FATAL... and abort
      ()s
#else
  std::cout << "NO_CHECK: cref() is just const int&, "
            << "type is int const&: "
            << std::is_same_v<decltype(m.cref()), const int&> << '\n';
#endif
}
```

The same source gives three different behaviours, selected only by the macro.

Listing 28: Compile with `-DMEMVAR_SAFEREF_CHECK=0` (abort mode); the process ends with signal 6.

```
after 2 more writes: 1
dereferencing the expired handle now aborts the process
FATAL: mv::memvar::cref(): the referenced value has been evicted from history
    (too many writes, or a clearHistory(), happened since cref() was called).
    Terminating.
```

Listing 29: Compile with `-DMEMVAR_SAFEREF_CHECK=1` (throwing mode).

```
after 2 more writes: 1
expired handle detected: mv::memvar::cref(): the referenced value has been
    evicted from history (too many writes, or a clearHistory(), happened since
     cref() was called)
```

Listing 30: Default build (`-DMEMVAR_SAFEREF_CHECK=2`): no handle exists.

```
after 2 more writes: 1
NO_CHECK: cref() is just const int&, type is int const&: 1
```

In modes 0 and 1 the handle is valid after two further writes (the history of capacity 3 still holds the value) and expires on the third, exactly as in Table 5. In mode 2 there is nothing to expire, and code that keeps the reference across three writes would be reading a destroyed element.

7.7 Strings and Unicode output

Listing 31: Printing wide and Unicode strings; output shown as UTF-8 bytes.

```
#include <iomanip>
#include <iostream>
#include <sstream>
#include "memvar.h"

static void hex(const std::string& s) {
  for (unsigned char c : s) std::cout << std::hex << std::setw(2) << std::
      setfill('0') << int(c) << ' ';
  std::cout << std::dec << "(" << s.size() << " bytes)\n";
}

int main() {
  // UTF-32 -> UTF-8: U+00E9 and U+1F600
  mv::memvar<std::u32string> u32(U"\u00e9\U0001F600");
  std::ostringstream a; a << u32;  hex(a.str());

  // UTF-16 surrogate pair is combined into one 4-byte UTF-8 sequence
  mv::memvar<std::u16string> u16(u"\U0001F600");
  std::ostringstream b; b << u16;  hex(b.str());

  // a lone surrogate becomes U+FFFD
  mv::memvar<std::u16string> bad(std::u16string(1, char16_t(0xD800)));
  std::ostringstream c; c << bad;  hex(c.str());

  // wstring on this platform (wchar_t is 32 bits on Linux)
  mv::memvar<std::wstring> w(L"\u20ac");
  std::ostringstream d; d << w;    hex(d.str());

  // u8string is forwarded byte for byte
  mv::memvar<std::u8string> u8(u8"\u00e9");
  std::ostringstream e; e << u8;   hex(e.str());

  // string history
  mv::memvar<std::string> s("a", 3);
  s = "b"; s = "c"; s = "d";
  s.printReverseHistoryData(std::cout, ",");
}
```

Listing 32: Output of Listing 31.

```
c3 a9 f0 9f 98 80 (6 bytes)
f0 9f 98 80 (4 bytes)
ef bf bd (3 bytes)
e2 82 ac (3 bytes)
c3 a9 (2 bytes)
[ b,c,d,]
```

The first line is U+00E9 followed by U+1F600 encoded as `c3 a9` and `f0 9f 98 80`; the UTF-16 surrogate pair becomes the same four bytes as one code point; a lone high surrogate becomes the replacement character `ef bf bd`; and `std::u8string` bytes pass through unchanged.

8 Edge Cases

The program below collects the compile-time properties (checked with `static_assert`, so a regression would break the build) and the run-time edge cases in one place.

Listing 33: Edge-case program: compile-time properties and run-time behaviour.

```
#include <cstdint>
#include <iostream>
#include <string>
#include <type_traits>
#include <vector>
#include "bigint.h"
#include "memvar.h"

using mv::memvar;
using mv::memvarTimed;

// ---- compile-time properties (all checked with static_assert) -------------
static_assert(!std::is_copy_constructible_v<memvar<int>>);
static_assert(!std::is_move_constructible_v<memvar<int>>);
static_assert(!std::is_move_assignable_v<memvar<int>>);            // deleted overload wins
static_assert(std::is_copy_assignable_v<memvar<int>>);             // value copy only
static_assert(!std::is_copy_constructible_v<memvarTimed<int>>);
static_assert(std::has_virtual_destructor_v<memvar<int>>);
// memvarTimed<int> is NOT a memvar<int>: its base is memvar<int, TimePolicy>
static_assert(!std::is_base_of_v<memvar<int>, memvarTimed<int>>);
static_assert(!std::is_convertible_v<memvarTimed<int>&, const memvar<int>&>);

template <class L, class R> concept HasPlusEq  = requires(L& l, const R& r) { l += r; };
template <class L, class R> concept HasMinusEq = requires(L& l, const R& r) { l -= r; };
template <class L, class R> concept HasMinus   = requires(const L& l, const R& r) { l - r; };
template <class L>          concept HasPreInc  = requires(L& l) { ++l; };
template <class L, class R> concept HasMod     = requires(const L& l, const R& r) { l % r; };
template <class L>          concept HasNeg     = requires(const L& l) { -l; };

static_assert( HasPlusEq <memvar<std::string>, const char*>);
static_assert(!HasMinusEq<memvar<std::string>, std::string>);
static_assert(!HasMinus  <memvar<std::string>, std::string>);
static_assert(!HasPreInc <memvar<std::string>>);
static_assert( HasPreInc <memvar<double>>);
static_assert( HasMod    <memvar<int>, int>);             // via implicit conversion to int
static_assert(!HasMod    <memvar<bip::bigint>, int>);     // member operator: no conversion
static_assert( HasNeg    <memvar<int>>);                  // via implicit conversion
static_assert(!HasNeg    <memvar<bip::bigint>>);

template <class T> concept Constructible = requires { typename memvar<T>; };

// result types of the binary operators (no re-narrowing to T)
```

```
43 static_assert(std::is_same_v<decltype(std::declval<memvar<std::int8_t>&>() +
       std::declval<memvar<std::int8_t>&>()), int>);
44 static_assert(std::is_same_v<decltype(std::declval<memvar<int>&>() + 1.5),
       double>);
45 static_assert(std::is_same_v<decltype(std::declval<memvar<int>&>() == std::
       declval<memvar<long>&>()), bool>);
46
47 template <class F> void attempt(const char* what, F f) {
48   try { f(); std::cout << what << ": no exception\n"; }
49   catch (const std::exception& e) { std::cout << what << ": " << typeid(e).
         name() << " -> " << e.what() << '\n'; }
50 }
51
52 int main() {
53   attempt("capacity 1", [] { memvar<int> m(0, 1); });
54   attempt("capacity 0", [] { memvar<int> m(0, 0); });
55   attempt("capacity -5", [] { memvar<int> m(0, -5); });
56   attempt("capacity 2 ", [] { memvar<int> m(0, 2); });
57
58   memvar<int> m(10, 3);
59   attempt("index -1", [&] { (void)m(-1); });
60   attempt("index 1 (size 1)", [&] { (void)m.valueAt(1); });
61   attempt("getHistoryValue(5)", [&] { (void)m.getHistoryValue(5); });
62   memvarTimed<int> t(1, 3);
63   attempt("timed getTimeTag(3)", [&] { (void)t.getTimeTag(3); });
64   attempt("timed getTimeTag(-1)", [&] { (void)t.getTimeTag(-1); });
65   attempt("timed getHistoryTimedValue(-1)", [&] { (void)t.getHistoryTimedValue
         (-1); });
66
67   // self assignment records a duplicate
68   m = m;
69   std::cout << "after self-assign size=" << m.getHistorySize() << " : ";
70   m.printHistoryData();
71
72   // compound assignment with a different arithmetic type truncates silently
73   memvar<int> q(1, 4);
74   q += 1.9;  std::cout << "int += 1.9  -> " << q << '\n';
75   memvar<std::int8_t> small(120, 3);
76   small += 20; std::cout << "int8 120+=20 -> " << int(small()) << '\n';
77
78   // ++ on a narrow type is computed in int and converted back
79   memvar<unsigned char> uc(255, 3);
80   uc++;        std::cout << "uchar 255++ -> " << int(uc()) << '\n';
81
82   // char-like T are printed as characters
83   memvar<char> ch('A', 3);  ++ch;
84   std::cout << "char: " << ch << '\n';
85
86   // clearHistory resets to T{}, not to the latest value
87   memvar<int> keep(42, 3); keep = 43; keep.clearHistory();
88   std::cout << "after clearHistory: " << keep << '\n';
89
90   // postfix returns the old value
91   memvar<int> p(5, 3);
92   int old = p++;
93   std::cout << "p++ returned " << old << ", p is " << p << '\n';
94
95   // auto cref: in NO_CHECK mode this is a COPY of the value (auto drops the
         reference)
96   auto c = p.cref();
97   std::cout << "auto cref is a handle type: " << !std::is_same_v<decltype(c),
       int> << '\n';
98
```

```
99    // mixed comparisons between different memvar types
100   memvar<long> l(7);
101   memvar<int> i7(7);
102   std::cout << "memvar<int>(7) == memvar<long>(7): " << (i7 == l) << '\n';
103   memvarTimed<int> ti(7);
104   std::cout << "memvar<int> == memvarTimed<int>: " << (i7 == ti) << '\n';
105
106   // bare memvar assigned to a timed one: goes through operator T()
107   ti = i7;
108   ti += i7;
109   std::cout << "timed after = and += from bare memvar: " << ti() << '\n';
110 }
```

Listing 34: Output of Listing 33 (default mode).

```
capacity 1: St16invalid_argument -> ERROR: The history capacity must be 2 at
    least, or more
capacity 0: St16invalid_argument -> ERROR: The history capacity must be 2 at
    least, or more
capacity -5: St16invalid_argument -> ERROR: The history capacity must be 2 at
    least, or more
capacity 2 : no exception
index -1: St12out_of_range -> Index -1 is out of bounds (size is 1).
index 1 (size 1): St12out_of_range -> Index 1 is out of bounds (size is 1).
getHistoryValue(5): St12out_of_range -> Index 5 is out of bounds (size is 1).
timed getTimeTag(3): St12out_of_range -> Index 3 is out of bounds (size is 1).
timed getTimeTag(-1): St12out_of_range -> Index -1 is out of bounds (size is
    1).
timed getHistoryTimedValue(-1): St12out_of_range -> Index -1 is out of bounds
    (size is 1).
after self-assign size=2 : [ 10 10 ]
int += 1.9  -> 2
int8 120+=20 -> -116
uchar 255++ -> 0
char: B
after clearHistory: 0
p++ returned 5, p is 6
auto cref is a handle type: 0
memvar<int>(7) == memvar<long>(7): 1
memvar<int> == memvarTimed<int>: 1
timed after = and += from bare memvar: 14
```

8.1 Capacity and index errors

- Capacities below 2 (including 0 and negative values) throw `std::invalid_argument` from the constructor, before any storage exists. Capacity 2 is accepted. The minimum is a fixed constant (`minimumHistoryCapacity_`).
- A value access with an index that is negative or not smaller than the current *size* (not the capacity) throws `std::out_of_range` with the message `Index` *i* `is out of bounds (size is` *n*`).`: a freshly constructed variable has size 1, so `m(1)` throws until a second value has been written.
- The time-tag accessors `getTimeTag` and `getHistoryTimedValue` report the same condition with the same message, including for negative indices (the run above shows `getTimeTag(3)`, `getTimeTag(-1)` and `getHistoryTimedValue(-1)`). In the former header they used the standard library's text, accepted a `size_t` and reported `-1` as 18446744073709551615.

8.2 Assignment edge cases

- **Self-assignment** (`m = m`) is not a no-op: it records the current value again (history `[ 10 10 ]`), pushing older values out. Since the deque keeps references to existing elements valid when an element is added at the front, constructing the new element from the old front element is safe.
- **Assigning from a bare `memvar` to a `memvarTimed`** and `+=` with a bare `memvar` on the right both work (they go through the conversion to `T` and through the generic operator, respectively): the program prints 14 after `ti = i7; ti += i7;` with `i7 == 7`.
- **Mixed value types** compare and add (`memvar<int> == memvar<long>`, `memvar<int> == memvarTimed<int>`), because the operator templates only require that the underlying operation exist.

8.3 Numeric edge cases

- **Silent truncation in compound assignment:** adding 1.9 to a `memvar<int>` holding 1 records 2; adding 20 to a `memvar<int8_t>` holding 120 records -116 (the sum is computed in `int` and converted back).
- **Narrow types and `++`:** `memvar<unsigned char>(255)++` records 0 (computed in `int`, converted back with `static_cast`); `memvar<char>('A')` increments to `'B'` and prints as a character.
- **Signed overflow and division by zero** for `int`-width types and wider are those of the built-in operators: `INT_MAX + 1` or `m /= 0` is undefined behaviour *(by inspection; not run)*. Only `bigint` reports division by zero as an exception (`std::invalid_argument`, seen in Listing 23).
- **Mixed signedness:** the operators forward to the built-in comparison, so `memvar<int>(-1) < memvar<unsigned>(1)` yields `false`, exactly like `-1 < 1u`. Since the operator body is now the plain expression `valueOf(lhs) < valueOf(rhs)` inside `memvar.h`, GCC does warn (`-Wsign-compare`); the warning points at the macro line in the header, and the user's own line appears only as "required from here":

Listing 35: Output and compiler diagnostic of the mixed-signedness probe.

```
memvar<int>(-1) < memvar<unsigned>(1): false
-1 < 1u (built-in): false
--- compiler diagnostic (g++ 13.3 -Wall -Wextra):
edge6_mixed.cpp:7:86:   required from here
memvar.h:523:24: warning: comparison of integer expressions of different
    signedness: 'const int' and 'const unsigned int' [-Wsign-compare]
edge6_mixed.cpp:8:46: warning: comparison of integer expressions of different
    signedness: 'int' and 'unsigned int' [-Wsign-compare]
```

8.4 Reference-handle edge cases

- In the default mode `auto c = m.cref();` deduces `int` and copies; the edge-case program prints `0` for "is a handle type". In modes 0 and 1 the same statement declares a `SafeRef` and prints `1`.
- `clearHistory()` expires every outstanding handle even though only a single write is logically involved.
- A handle is a view of a *slot*: `h = m.cref(); m = 99;` leaves `h` designating the previous value.
- A write that throws does not count towards the expiry of a handle (modes 0 and 1; covered by the unit test `DoesNotAgeOutstandingSafeRefs`).

8.5 Failed writes: the strong guarantee

The program below makes a write fail deterministically, without limiting the address space of the process: the global `operator new` is replaced by a version that throws `std::bad_alloc` for the next allocation of at least 1 000 bytes once armed (the 5 001-byte string copy; the deque nodes are far smaller). It then checks that the value and time histories of a `memvarTimed<std::string>` still have the same length.

Listing 36: Reproducing a failed write on a memvarTimed.

```
// Reproduces: a failed write leaves memvarTimed's value history and time history out of step.
// Build: g++ -std=c++20 -O1 -I<dir with memvar.h> repro_timed_failed_write.cpp -o repro
// No ulimit needed: a replaced global operator new fails on demand.
#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <new>
#include <string>
#include "memvar.h"

static bool g_failNextBigAlloc = false;            // armed just before the write
void* operator new(std::size_t n) {
  if (g_failNextBigAlloc && n >= 1000) {           // only the string copy is that large
    g_failNextBigAlloc = false;
    throw std::bad_alloc();
  }
  if (void* p = std::malloc(n)) return p;
  throw std::bad_alloc();
}
void operator delete(void* p) noexcept { std::free(p); }
void operator delete(void* p, std::size_t) noexcept { std::free(p); }

int main() {
  mv::memvarTimed<std::string> m("first", 10);
  const std::string big(5000, 'x');                // allocated BEFORE arming the failure

  std::cout << "size before failed write : " << m.getHistorySize() << '\n';

  g_failNextBigAlloc = true;
  try {
    m = big;                                       // copy -> allocation -> bad_alloc
    std::cout << "write unexpectedly succeeded\n";
  } catch (const std::bad_alloc&) {
    std::cout << "write failed with bad_alloc\n";
  }
  g_failNextBigAlloc = false;

  std::cout << "size after failed write  : " << m.getHistorySize() << '\n';

  // With a consistent object, getTimeTag(1) must throw: only index 0 exists.
  try {
    (void)m.getTimeTag(1);
    std::cout << "BUG: getTimeTag(1) succeeded, time history is longer than value history\n";
  } catch (const std::out_of_range&) {
    std::cout << "getTimeTag(1) threw (consistent)\n";
  }

```

```
48    m = "second";                                          // a normal write
49    std::cout << "after next good write: value(0)=" << m(0) << " value(1)=" << m
          (1) << '\n';
50    try {
51      (void)m.getTimeTag(2);
52      std::cout << "BUG: getTimeTag(2) succeeded but the value history has only
            "
53                << m.getHistorySize() << " entries\n";
54    } catch (const std::out_of_range&) {
55      std::cout << "getTimeTag(2) threw (consistent: " << m.getHistorySize() <<
            " entries)\n";
56    }
57  }
```

Listing 37: Output of Listing 36 against the current header.

```
size before failed write : 1
write failed with bad_alloc
size after failed write  : 1
getTimeTag(1) threw (consistent)
after next good write: value(0)=second value(1)=first
getTimeTag(2) threw (consistent: 2 entries)
```

Against the former header (two deques, time point recorded before the value was constructed) the same source prints:

Listing 38: Output of Listing 36 against the former header.

```
size before failed write : 1
write failed with bad_alloc
size after failed write  : 1
BUG: getTimeTag(1) succeeded, time history is longer than value history
after next good write: value(0)=second value(1)=first
BUG: getTimeTag(2) succeeded but the value history has only 2 entries
```

After the failed write the former header's value history had one element, yet `getTimeTag(1)` succeeded, which is impossible if the invariant holds; after the next successful write the time tag that belonged to the *failed* write was paired with the previous value. With the current header the failed write is invisible: the size is unchanged, `getTimeTag(1)` throws, and after the next good write the entry of `"first"` still carries its own tag. The unit tests of Section 9 turn this program into assertions for plain and timed variables, for full histories and for `clearHistory()`.

8.6 Consequences of the single-deque design

The following program checks, partly at compile time, the properties that follow from `memvarTimed` not being a `memvar<T>`: it is not accepted where `const memvar<int>&` is expected, but is accepted by a function template over `memvar<T, P>`; the history container of a timed variable holds `Stamped` entries; the member `getHistoryTimedValue` keeps the tag while the free function of the same name returns the value only; and `clearHistory` moves the epoch.

Listing 39: Edge cases of the single-deque design.

```
1  // Consequences of the single-deque design (memvar<T, Policy>).
2  #include <chrono>
3  #include <iostream>
4  #include <string>
5  #include <type_traits>
6  #include <typeinfo>
7  #include <thread>
```

```cpp
#include "memvar.h"

using namespace std::chrono_literals;
using mv::memvar;
using mv::memvarTimed;

// A function written against the plain class ...
int previous(const memvar<int>& m) { return m(1); }
// ... and one that takes any memvar flavour.
template <class T, class P> int previousAny(const memvar<T, P>& m) { return m(1); }

template <class X> concept AcceptedByPlainRef = requires(X& x) { previous(x); };

static_assert( AcceptedByPlainRef<memvar<int>>);
static_assert(!AcceptedByPlainRef<memvarTimed<int>>);   // no longer a memvar<int>
static_assert(std::is_same_v<decltype(std::declval<const memvar<int>&>().getMemVarHistory()),
                             const std::deque<int>&>);
static_assert(std::is_same_v<decltype(std::declval<const memvarTimed<int>&>().getMemVarHistory()),
                             const std::deque<mv::detail::Stamped<int, std::chrono::steady_clock::time_point>>&>);
static_assert(std::is_same_v<memvarTimed<int>::Slot,
                             mv::detail::Stamped<int, std::chrono::steady_clock::time_point>>);
static_assert(std::is_same_v<decltype(mv::getHistoryTimedValue(std::declval<const memvarTimed<int>&>(), 0)), int>);
static_assert(std::is_same_v<decltype(std::declval<const memvarTimed<int>&>().getHistoryTimedValue(0)),
                             std::tuple<int, std::chrono::nanoseconds, bool>>);

int main() {
  std::cout << "sizeof(Stamped<int>)            = "
            << sizeof(memvarTimed<int>::Slot) << '\n';

  memvarTimed<int> t(10, 4);
  std::this_thread::sleep_for(2ms);
  t = 20;
  std::this_thread::sleep_for(2ms);
  t = 30;

  std::cout << "previousAny(t)                  = " << previousAny(t) << '\n';

  // the history container of a timed variable holds {value, tag} pairs
  std::cout << "entries (value, tag >= 2 ms after the oldest tag):";
  const auto oldest = t.getMemVarHistory().back().tag;
  for (const auto& s : t.getMemVarHistory())
    std::cout << " (" << s.value << ", " << (s.tag - oldest >= 2ms) << ")";
  std::cout << '\n';

  std::cout << "first value has tag exactly zero: " << (t.getTimeTag(2) == std::chrono::nanoseconds::zero()) << '\n';

  // the free function drops the tag, the member keeps it
  auto [v, tag, err] = t.getHistoryTimedValue(1);
  std::cout << "member getHistoryTimedValue(1)  -> value " << v << ", tag>=2ms: "
            << (tag >= 2ms) << ", err " << err << '\n';
  std::cout << "free   getHistoryTimedValue(t,1) -> " << mv::
```

```
      getHistoryTimedValue(t, 1) << " (value only)\n";

  // clearHistory moves the epoch to the new single entry
  t.clearHistory();
  std::cout << "after clearHistory: size " << t.getHistorySize()
            << ", getTimeTag(0) = " << t.getTimeTag(0).count() << " ns, value
                " << t() << '\n';

  // min/max works on the value part only
  memvarTimed<int> r(5, 4); r = 1; r = 9;
  auto [mn, mx] = r.getHistoryMinMax();
  std::cout << "getHistoryMinMax -> " << mn << " " << mx << '\n';
}
```

Listing 40: Output of Listing 39.

```
sizeof(Stamped<int>)             = 16
previousAny(t)                   = 20
entries (value, tag >= 2 ms after the oldest tag): (30, 1) (20, 1) (10, 0)
first value has tag exactly zero: 1
member getHistoryTimedValue(1)  -> value 20, tag>=2ms: 1, err 0
free   getHistoryTimedValue(t,1) -> 20 (value only)
after clearHistory: size 1, getTimeTag(0) = 0 ns, value 0
getHistoryMinMax -> 1 9
```

8.7 Stream operator lookup

Listing 41: Name lookup of operator« inside a namespace that declares its own.

```
#include <iostream>
#include <sstream>
#include "memvar.h"
namespace app {
struct Point { int x; };
std::ostream& operator<<(std::ostream& os, const Point& p) { return os << p.x;
    }
std::string viaApp(const mv::memvarTimed<int>& m) { std::ostringstream o; o <<
    m; return o.str(); }
std::string viaAppW(const mv::memvar<std::wstring>& m) { std::ostringstream o;
#ifdef TRY_W
  o << m;
#endif
  return o.str(); }
}
int main() {
  mv::memvarTimed<int> m(3);
  std::ostringstream o; o << m;
  std::cout << "global scope : " << o.str() << '\n';
  std::cout << "inside app   : " << app::viaApp(m) << '\n';
}
```

Listing 42: Output of Listing 41 (without TRY_W).

```
global scope : [0:3]
inside app   : 3
```

Inside namespace app, which declares an operator« for an unrelated type, the global operator« for memvarTimed is hidden. The expression still compiles for an int payload because the implicit operator T() converts the object to int: the output silently changes from [0:3] to 3. With -DTRY_W, the same

statement for a memvar<std::wstring> does not compile ("no match for operator«"), because there is no std::wstring insertion visible from that scope either.

8.8 Unusable but accepted types

As Table 2 showed, const T and reference types satisfy the concept but fail inside the standard library. bool and std::string_view are rejected cleanly ("template constraint failure").

8.9 Containers and storage of memvars

Because copy and move construction are deleted, a memvar cannot be returned by value from a function that does not elide it, cannot be an element of a std::vector whose storage may be reallocated, and has no swap. Node-based containers that construct elements in place (std::list::emplace_back, std::deque::emplace_back, std::map::try_emplace) and std::unique_ptr<memvar<T>> are the usable options *(by inspection)*.

9 The Unit-Test Suite

9.1 Structure and results

The project's unitTests.cpp (1 525 lines) uses GoogleTest; it includes bigint.h before ../memvar.h, as required by the include-order rule of Section 6, and defines 49 TESTs in seven suites (Table 10). Most tests are written in a direct style: perform a sequence of writes, then assert the indexed values, sizes, capacities and comparison results; many also print the history to the standard streams, but no test captures or checks printed text.

Suite	Tests	What it covers
memVarTimedTest	13	the timed variable: ordered writes and indexing, constructor validation for int, double, string, bigint, capacity 2 with getHistoryMinMax, default construction, increments and compound operators, a full history and clearHistory, strings, char with ++/-, comparisons between timed variables, the tuple accessors, timing loops, Fibonacci with int64_t and bigint
memVarTest	17	the same list for the plain variable, plus crefTest (three #if branches, one per safety mode, including EXPECT_THROW and EXPECT_DEATH), passing by reference, unique_ptr and shared_ptr owners, and two stress tests (test_6: a history of 10^9 entries; test_7: 10^9 writes with capacity 100)
MemvarTimedAssignFromBare Memvar	3	timed = plain compiles and gives the right value, also for std::string, although the base-class operator= is hidden
MemvarTimedCompoundAssign FromBareMemvar	6	timed += plain etc. are well-formed (checked with requires-expression concepts) and give the right values
MemvarFailedWrite	5	failed writes on plain variables
MemvarTimedFailedWrite	3	failed writes on timed variables
MemvarTimedEntries	2	values and tags are paired; tag index checking

Table 10: The seven test suites.

The tests were built and run in each safety mode (-O3 -DNDEBUG -Wall -Wextra -pedantic -march=native); memVarTest.test_6 was excluded in every run, because it allocates about 8 GB.

The sanitizer runs used the following options, with AddressSanitizer's allocation/deallocation-mismatch check switched off (ASAN_OPTIONS=alloc_dealloc_mismatch=0), because the test file replaces operator new and operator delete with malloc/free-based versions. No report was produced.

Mode	Defined	Run	Passed	Remark
0 (abort)	49	48	48	also under ASan+UBSan
1 (throws)	49	48	48	also under ASan+UBSan
2 (no check)	48	47	47	`DoesNotAgeOutstandingSafeRefs` is compiled out; also under ASan+UBSan

Table 11: Unit-test results against the delivered header.

Listing 43: Sanitizer build options.

```
-O1 -g -fsanitize=address,undefined -fno-sanitize-recover=undefined
```

9.2 Injecting an allocation failure

A failed write cannot be provoked portably with real memory pressure, so the new tests replace the global allocation function. The relevant part of the file is shown in Listing 44.

Listing 44: Failure injection in unitTests.cpp.

```
#include <cstdlib>
#include <new>

namespace failinj {
inline bool armed = false;
inline constexpr std::size_t threshold = 1000;
}  // namespace failinj

// The replacement pair is malloc/free based, which is valid; but at -O2/-O3 GCC
// inlines operator new/delete into callers (e.g. std::make_unique) and then
// reports -Wmismatched-new-delete because it sees free() applied to a pointer
// that came from operator new. The warning is a false positive for a matched
// replacement pair, so it is silenced for these definitions only.
#if defined(__GNUC__) && !defined(__clang__)
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wmismatched-new-delete"
#endif
void* operator new(std::size_t n) {
  if (failinj::armed && n >= failinj::threshold) {
    failinj::armed = false;               // fail exactly once
    throw std::bad_alloc();
  }
  if (void* p = std::malloc(n ? n : 1)) return p;
  throw std::bad_alloc();
}
void operator delete(void* p) noexcept { std::free(p); }
void operator delete(void* p, std::size_t) noexcept { std::free(p); }
#if defined(__GNUC__) && !defined(__clang__)
#pragma GCC diagnostic pop
#endif

namespace {

// Runs `write` with the next large allocation made to fail; returns true if it
// threw std::bad_alloc.
template <typename F>
bool writeThrowsBadAlloc(F&& write) {
  bool threw = false;
  failinj::armed = true;
```

```
  try {
    write();
  }
  catch (const std::bad_alloc&) {
    threw = true;
  }
  failinj::armed = false;
  return threw;
}

const std::string bigString(5000, 'x');    // built before any failure is armed

}  // namespace
```

- `failinj::armed` is a flag; while it is set, the first allocation of at least `failinj::threshold` (1 000) bytes throws `std::bad_alloc` and clears the flag, so exactly one allocation fails. Deque nodes (512 bytes), the strings used as values and everything GoogleTest allocates are smaller than the threshold; only the copy of `bigString` (5 001 bytes with the terminator) is larger.
- `writeThrowsBadAlloc(f)` arms the flag, runs one write and reports whether `std::bad_alloc` was thrown; the flag is cleared again on every exit path.
- `bigString` is a namespace-scope constant, constructed before any test arms the flag.
- `InjectionWorks` asserts that the hook fires at all, so that the other tests cannot pass vacuously.
- The replacement `operator delete` calls `std::free`. At `-O2` and `-O3` GCC inlines the replaced `operator new` into callers such as `std::make_unique` and then reports `-Wmismatched-new-delete`, which is a false positive for a matched replacement pair. The file silences it with `#pragma GCC diagnostic ignored` around these three definitions only, guarded so that other compilers do not see it. The warning was observed with GCC 16.2.1 and reproduced with g++ 13 and 14.

9.3 The new tests

The last column was obtained by compiling the same `unitTests.cpp` against the former two-deque header: 43 of 46 tests passed and exactly the three marked **yes** failed. The tests on plain variables pass on both headers because the plain write path was already correct (the single `emplace_front`); the defect was confined to the timed class. A representative test is shown in Listing 45.

Listing 45: HistoriesStayInStep: the regression test for the defect.

```
TEST(MemvarTimedFailedWrite, HistoriesStayInStep) {
  mv::memvarTimed<std::string> m("first", 10);
  const auto tag0 = m.getTimeTag(0);

  ASSERT_EQ(m.getHistorySize(), 1);
  EXPECT_TRUE(writeThrowsBadAlloc([&] { m = bigString; }));

  // value history and time history both unchanged
  EXPECT_EQ(m.getHistorySize(), 1);
  EXPECT_EQ(m(), "first");
  EXPECT_EQ(m.getTimeTag(0), tag0);
  EXPECT_THROW((void)m.getTimeTag(1), std::out_of_range);    // used to succeed
  EXPECT_THROW((void)m.getHistoryTimedValue(1), std::out_of_range);

  // a later good write pairs each value with its own tag
  m = "second";
  EXPECT_EQ(m.getHistorySize(), 2);
```

Test	Scenario and assertions	Fails on former header
`MemvarFailedWrite.InjectionWorks`	the hook throws, and disarms itself	no
`.PlainMemvarIsUnchanged`	failed write on `memvar<string>`: size, both values, `m(2)` still throws; a later write works	no
`.FullHistoryDoesNotEvictTheOldestValue`	capacity 2, full: the failed write keeps the oldest value	no
`.CompoundAssignmentIsUnchanged`	a failing `m += bigString` changes nothing	no
`.DoesNotAgeOutstandingSafeRefs` (modes 0, 1)	a failed write does not count; the handle is still valid after the next good write	no
`MemvarTimedFailedWrite.HistoriesStayInStep`	after the failed write: size 1, same value, same tag, `getTimeTag(1)` and `getHistoryTimedValue(1)` throw; after a good write each value keeps its own tag	**yes**
`.FullHistoryKeepsValuesAndTagsTogether`	full history of 3: nothing changes after the failure; a later write evicts value and tag together	**yes**
`.AfterClearHistory`	after `clearHistory()` the tag is exactly zero and survives a failed write	**yes**
`MemvarTimedEntries.ValueAndTagAreAlwaysPaired`	9 writes at 200 μs spacing into capacity 4: for each index, `getHistoryTimedValue(i)` agrees with `valueAt(i)` and `getTimeTag(i)`; newer tags are larger	no
`.TimeTagIndexIsCheckedLikeValueIndex`	`getTimeTag(-1)`, `getTimeTag(1)`, `getHistoryTimedValue(-1)` throw `out_of_range`; `getTimeTag(0)` does not	no (only the message differs)

Table 12: New tests and their ability to detect the defect of the former header.

```
1412    EXPECT_EQ(m(0), "second");
1413    EXPECT_EQ(m(1), "first");
1414    EXPECT_EQ(m.getTimeTag(1), tag0);          // tag of "first", not of the
            failed write
1415    EXPECT_GE(m.getTimeTag(0), m.getTimeTag(1));
1416    EXPECT_THROW((void)m.getTimeTag(2), std::out_of_range);
1417
1418    const auto [v1, t1, e1] = m.getHistoryTimedValue(1);
1419    EXPECT_EQ(v1, "first");
1420    EXPECT_EQ(t1, tag0);
1421    EXPECT_FALSE(e1);
1422 }
```

9.4 Gaps and fragilities of the suite

These points follow from the test code; none is a defect of `memvar.h`.

- **Not covered:** output text (no test captures a stream, so the UTF-8 helpers, the `[ ... ]` and `[tag:value]` formats are untested), `getMemVarHistory()`, the wide and Unicode string types, mixed-type operands, a non-default `Time` or `Clock` (used by one new test only), and any behaviour of the constraint system (rejected types are compile errors and cannot be tested by a compiled test).
- **No thread-safety tests**, consistent with the stated assumption that the header is not thread-safe.
- **Timing dependence.** `ValueAndTagAreAlwaysPaired` compares tags with a strict inequality after a 200 μs sleep and uses microsecond resolution; it relies on `steady_clock` advancing by at least a microsecond over a sleep, which holds on the test machine.

- **Cost.** `test_7` performs 10^9 writes and takes most of the suite's 3 s; `test_6` needs gigabytes of memory.
- **Program-wide replacement of `operator new`.** The injection hook applies to the whole test binary; the 1 000-byte threshold is an unexplained constant that would need revisiting if another test allocated a large object while the hook is armed (it is armed only around one statement, so none does).
- **Compiler warnings in the test file itself.** With `-Wall -Wextra`, `crefTest` produces `-Wunused-value` for `EXPECT_THROW(*xcref + 1, ...)` (modes 0 and 1, on line 594 or 599 depending on the mode) and `-Wunused-but-set-variable` for `xcrefCopy` (modes 0 and 1). The default mode builds without warnings.

10 Limitations, Constraints, and Open Issues

This section separates *constraints* (deliberate restrictions that a user must know), from *open issues* (behaviours that are, in the view of this analysis, defects or traps). The evidence column of Table 13 says whether an item was **reproduced** by a program in this report, **measured**, or derived **by inspection** of the code only. The suggested directions are proposals; none has been implemented in the analysed header (the one change that was made, the single-deque history of `memvarTimed`, resolved issue I1 and is described in Section 4.6).

10.1 Constraints of the design

- **Value types** are limited to non-`bool` integrals, floating point, the five standard string types and `bip::bigint` (Section 2). The only extension point is specialising `mv::is_bigint`, and a new type must also provide the operators, the default constructor and `operator«` the header uses.
- **Capacity** is fixed per object at construction, at least 2, and stored as a signed 64-bit number.
- **Write guarantee:** every write (including compound assignment and `clearHistory`) has the strong exception guarantee, provided the copy/move constructor of `T` either succeeds or throws without side effects.
- **Type relationship:** `memvarTimed<T,...>` is not a `memvar<T>`; code that must accept both is written as a template over `memvar<T, P>` or over the operators.
- **No copying or moving** of a variable; value-only copy assignment.
- **Not thread-safe**: neither the deque, the counters nor the time history are synchronised. This is a stated assumption, not a defect.
- **History semantics:** index 0 is the newest value; history length is `min(writes, capacity)`; time tags are relative to a per-object epoch, which is the tag of the first value (reset by `clearHistory`).
- **C++20 required**; only g++ 13.3 (and, for one warning, g++ 14) with libstdc++ was exercised.
- **Include order:** `bigint.h` before `memvar.h`.
- **One value of `MEMVAR_SAFEREF_CHECK` per program.**

10.2 Summary of open issues

Table 13: Open issues and their evidence.

ID	Issue	Severity	Evidence	Where
I1	A failed write desynchronised the two histories of `memvarTimed`	**resolved**	reproduced on the former header, fixed and tested	§8.5
I2	Default mode gives no protection; `cref()` has a different type per mode	high	reproduced	§3.8
I3	Class layout depends on a macro (ODR)	medium	measured sizes	§6
I4	Arithmetic: overflow/division UB, silent truncation, mixed signedness (now with a warning inside the header)	medium	reproduced / inspection	§8
I5	`deque` storage: 3 allocations per object, block churn on writes (worse for timed entries)	medium	measured	§6
I6	Implicit `operator T()` gives an inconsistent operator set and hidden copies	medium	reproduced	§5
I7	`operator«` is global: lookup hides it, output format changes silently	medium	reproduced	§8
I8	Concept accepts types the class cannot hold	low	reproduced	§2
I9	Inconsistent accessor names/return types; vestigial error flag; free `getHistoryTimedValue` drops the tag	low	reproduced	§3
I10	`clearHistory` resets to `T{}`; fixed capacity; no snapshot	low	inspection	§3
I11	Not copyable/movable: unsuitable for `vector`, no `swap`	low	inspection	§8
I12	Hard-wired `bigint` coupling and duplicate trait	low	inspection	§2
I13	Time-tag semantics (epoch, tag taken before the value is built, clock choice, truncation, unit-less print)	low	inspection	§4
I14	`SafeRef` limits (owner lifetime, deduction contexts, cost of throwing mode)	low	inspection	§3.8
I15	Global-namespace Unicode stream overloads	low	inspection	§5
I16	Verification limited to one toolchain; one unit test not runnable here	info	this report	§6
I17	`memvarTimed` is no longer a `memvar<T>`; implementation types leak (`Stamped`, `Slot`); policy parameter unconstrained	medium	reproduced	§8.6

10.3 Details

I1 – RESOLVED: a failed write left `memvarTimed` inconsistent

Former behaviour. The timed override pushed a time point on a second deque and trimmed it. Only afterwards was the value constructed. If that construction threw (allocation failure, a throwing copy constructor), the time history already contained an extra entry, and every later pairing of value and tag (printing, `getHistoryTimedValue`) was wrong for the rest of the object's life, without any

exception or flag revealing it.
Resolution. Value and tag are one `Stamped` entry in one deque; a write is one `emplace_front` followed by steps that cannot throw (Section 3.4). `clearHistory` inserts before it erases.
Evidence. Listing 36 prints the defect against the former header and a consistent state against the current one; three new unit tests fail on the former header and pass on the current one (Section 9).
Residual note. The guarantee is as strong as the copy/move constructor of `T`: a type whose copy constructor has side effects is outside it. The tag is read before the value is constructed (issue I13).

I2 – Default mode is unsafe, and `cref()` changes type with the mode

What happens. In the default mode `cref()` is a plain `const T&` into a deque slot that is destroyed after `capacity` further writes; using it afterwards is undefined behaviour with no diagnostic. In the checked modes the same call returns a `SafeRef`. Code that is written against one mode can compile and behave differently under another: `auto c = m.cref();` is a *copy* (safe) in the default mode but a *handle that can expire* in modes 0/1, and `decltype(m.cref())` differs.
Evidence. Listing 27 with its three outputs; Listing 33 prints `0` for "is a handle type" in the default mode.
Direction. Document that, in the default mode, any reference obtained from `cref()` is valid only until `capacity` further writes and that `auto` copies; or return a handle type in every mode whose check compiles to nothing in mode 2.

I3 – The macro changes the class layout (ODR)

`writeCount_` and the nested `SafeRef` exist only in modes 0/1, so `sizeof(memvar<int>)` is 96 (mode 2) versus 104 (modes 0, 1) bytes, and `memvarTimed<int>` 104 versus 112. A program that links translation units compiled with different values has two different definitions of the same class template: undefined behaviour (ill-formed, no diagnostic required), typically observed as memory corruption. **Direction:** place each mode in a distinct inline namespace so that mixing produces a link error instead of silent corruption.

I4 – Arithmetic semantics are those of the built-in operators

- Signed overflow in `+ - *`, `++`, `-` and division or remainder by zero for integral `T` are undefined behaviour *(by inspection)*; `bigint` instead throws on division by zero.
- Compound assignment narrows silently: `memvar<int> += 1.9` records 2, `memvar<int8_t>(120) += 20` records −116 (reproduced).
- The binary operators return the *promoted* type (`int8_t + int8_t` is `int`), and permit mixed-type operands, so `memvar<int>(-1) < memvar<unsigned>(1)` is `false`; the compiler's `-Wsign-compare` fires, but inside `memvar.h` (reproduced), which makes it easy to suppress by mistake or to attribute to the library.

Direction: constrain compound assignment to results convertible to `T` without narrowing, and consider requiring both operands to have the same value type.

I5 – Storage cost and allocation behaviour of `std::deque`

With libstdc++ (512-byte nodes) a freshly built `memvar<int>(0,10)` makes 3 heap allocations and about 1 KiB, whatever the capacity; since the single-deque redesign a `memvarTimed<int>` makes the same 3 (it used to make 5 and 1.6 KiB). In steady state, *every* 128th write of an `int` (64th of a `long long`, 32nd of a timed `int`, whose entry is 16 bytes) frees a node and allocates another: 10^6 writes cost 7 812 (resp. 15 625 and 31 250) allocations even though the history never grows. The virtual call that every write used to make is gone. The measured write cost (2.4 ns) is equal to a hand-written deque push/pop, so the overhead is that of the container choice, not of the wrapper.

`getMemVarHistory()` additionally fixes `std::deque` as part of the public interface. **Direction:** a fixed-size ring buffer (one allocation, no churn, `capacity` known at construction) would remove both costs.

I6 – The implicit `operator T()` makes the operator set inconsistent

Because of the non-explicit conversion, operators that the header does not define still compile for built-in `T`: `m % 3`, `-m`, bitwise operators, shifts (they act on a temporary copy, record nothing, and have no compound forms; `%` and unary minus were verified, the others follow by inspection). For `bigint`, whose `%` and unary minus are *member* operators, the same expressions do not compile, because member lookup is not helped by conversions (reproduced with the concepts `HasMod` and `HasNeg` in Listing 33). The conversion also copies: `std::string s = m;` copies the whole string, and all of `operator()`, `operator[]`, `valueAt` and `getHistoryValue` return by value, which matters for large strings and bigints; the only copy-free read is `cref()` or `getMemVarHistory()`. **Direction:** declare the missing operators explicitly or make the conversion `explicit`; offer reference-returning accessors.

I7 – The stream operators are in the global namespace

Argument-dependent lookup cannot find them, so in any scope that sees another `operator«` first (for instance a namespace that overloads it for its own types) they are hidden. For `memvar<int>` the code still compiles through the implicit conversion and prints the bare value; for `memvarTimed<int>` the output silently changes from `[0:3]` to `3`; for `memvar<std::wstring>` compilation fails (Listing 41). **Direction:** define them as hidden friends or in namespace `mv`.

I8 – `MemVarType` accepts `const T` and references

The concept strips cv-qualifiers and references before testing, but the class stores `std::deque<T>` with the original `T`. `memvar<const int>` and `memvar<int&>` pass the constraint and then fail with errors inside `<bits/stl_deque.h>` (reproduced). The error text does not mention `memvar`. **Direction:** also require `std::same_as<T, std::remove_cvref_t<T>»`.

I9 – Inconsistent accessor names, results and diagnostics

- `memvar::getHistoryValue(i)` returns `tuple<T,bool>` while `mv::getHistoryValue(m, i)` returns `T`.
- The `bool` of the tuple is always `false`: out-of-range indices throw, so the "error flag" carries no information.
- `memvarTimed::getHistoryTimedValue(i)` returns the tuple with the time tag; `mv::getHistoryTimedValue(mvt, i)` returns only the value (it calls `mvt(i)`).
- `valueAt`, `operator()(i)` and `operator[]` are three spellings of one operation (the index-error messages of the value and tag accessors are now identical, which removes the former inconsistency).

I10 – Clearing and capacity

`clearHistory()` resets the *current value* to `T{}`; there is no way to drop the history but keep the latest value, other than writing it again, and the `SafeRef` counter is bumped at the same time (the call itself is now exception-safe: it inserts before it erases). The capacity cannot change after construction and cannot be less than 2; a capacity of 1 would be a meaningful degenerate case. There is no way to take a snapshot of the history as a value other than copying `getMemVarHistory()`.

I11 – Not copyable, not movable

Deleted copy/move construction and move assignment prevent `std::vector<memvar<T>>`, returning a named local, swapping, and aggregating memvars into a copyable struct. The value-only copy assignment may surprise: `a = b` between two variables of different capacity records one value and keeps `a`'s capacity.

I12 – Coupling to `bip::bigint`

The header hard-codes the name `bip::bigint` (forward declaration and trait specialisation), duplicates the trait that already exists as `bip::is_bigint`, and is correct only if `bigint.h` was included first. A different big-integer type can be admitted only by editing the header or specialising `mv::is_bigint`, and then must supply operator overloads that match the generic ones (for example `T + int`, used by ++).

I13 – Time-tag semantics

The epoch is the tag of the first entry, which therefore reads as exactly zero. Tags are offsets from a per-object epoch, so they are not comparable between objects or across a `clearHistory()`. The clock is read before the value is constructed, so for a large `T` the tag marks the start of the write rather than its commit. `Time` truncates (milliseconds lose sub-millisecond differences); a non-monotonic `Clock` can yield decreasing tags; printed tags carry no unit.

I14 – Limits of `SafeRef`

It protects only against slot eviction, not against destruction of the owner (raw pointer); implicit conversion fails in contexts that need template deduction or a member operator, so `*h` is needed there; `ABORT` mode terminates the whole process on a stale handle (appropriate for a test or debug configuration, hard to recover from in production); `THROWS` mode reintroduces the unwinding overhead the other mode avoids.

I15 – Overloads injected into the global namespace

The wide-string `operator«` templates are in the global namespace and apply to *every* `std::ostream`, not just to the header's own use. A program or another library that defines the same overloads gets ambiguity; invalid code units are silently replaced by U+FFFD, and there are no overloads for wide output streams.

I16 – Verification limits of this report

Only g++ 13.3/libstdc++ was used for the measurements (g++ 14 only to reproduce a diagnostic of the test file); sizes and allocation counts are specific to it. GCC 16, `clang++`, MSVC and other standard libraries were not available. The unit test `memVarTest.test_6` (one-billion-entry history) was not run. Undefined-behaviour statements in I4 were derived from the code, not triggered. AddressSanitizer and UBSan were run on the unit tests and examples with no report; ThreadSanitizer was not used, since thread safety is out of scope.

I17 – Consequences of `memvar<T, Policy>`

- `memvarTimed<int>` no longer converts to `const memvar<int>&` (checked with `static_assert`, Listing 39); a function written for the plain class cannot be handed a timed variable, and `memvarTimed<int, ms>` and `memvarTimed<int, ns>` do not share a base either. Under the previous design all of them were `memvar<int>`.
- `getMemVarHistory()` and the public alias `Slot` expose `mv::detail::Stamped`, so a name from a `detail` namespace is part of the user-visible interface of the timed class.

- The second template parameter is an unconstrained `typename`: `memvar<int, int>` fails with a hard error about the missing `tag_type` instead of a constraint failure, and the contract of a policy (a `tag_type` and a static `now()`) is written nowhere in code.
- `clearHistory` remains virtual, and the destructor, so the classes keep a vtable pointer although nothing but `clearHistory` is overridden.

Direction: add a `concept` for the policy; either give both flavours a common non-template base that carries the read-only interface, or document that generic code takes `memvar<T, P>`; move `Stamped` out of `detail` or hide it behind an accessor that returns value/tag views.

11 Conclusion

`memvar.h` implements a small, coherent idea: a variable that keeps its own bounded past, with ordinary-looking syntax. The current implementation is compact: one write path, one entry type chosen by a policy parameter so that `memvarTimed` reuses it without any hook, a single constrained template per operator, and a build-time switch that turns a cheap generation-counter check against dangling history references on or off. Measurements show that the wrapper adds no measurable cost over the deque it wraps, that the checked modes cost nothing noticeable in this benchmark, and that the move from two deques to one stamped deque left the speed unchanged while saving memory.

The most important defect of the previous version, a failed write that desynchronised the value and time histories of `memvarTimed`, is resolved by construction, and the new unit tests, which inject an allocation failure, pin the behaviour down and fail on the old code. The remaining weaknesses are at the edges: the default build's lack of protection together with a mode-dependent `cref()` type, a class layout that depends on a macro, an operator surface that mixes explicit templates with implicit conversions, stream operators that sit in the global namespace, a storage choice (`std::deque`) whose allocation behaviour is worse than the problem requires, and, new with the redesign, a timed class that is no longer a `memvar<T>`. If changes were prioritised, the order suggested by severity and cost would be (1) tighten `MemVarType` (I8) and put the safety modes in distinct inline namespaces (I3), (2) decide how generic code should refer to both flavours and constrain the policy (I17), (3) move the stream operators into namespace `mv` (I7), (4) replace the deque by a ring buffer (I5), and (5) decide whether the implicit conversion should remain (I6).